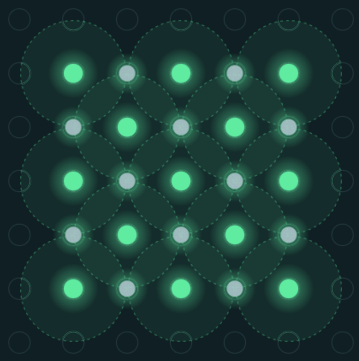
LEARNING GUIDE · PASQAL PLATFORM · OCTOBER 2026

Learn Pasqal, from a single atom to a cloud QPU



A step-by-step guide to **neutral-atom quantum computing** and the platform Pasqal has built around it. It starts with what a qubit is, moves through the physics of trapped atoms and Rydberg interactions, and ends with you writing **Pulser** programs, running them on emulators and sending them to real hardware through **Pasqal Cloud**. No physics degree needed.

Independent guide. Written for learners; not produced, reviewed or endorsed by Pasqal. Facts were checked against docs.pasqal.com, pasqal.com and a live install of Pulser 1.9.1 in October 2026. Software moves fast, so always confirm details in the official docs.

1,024	100–200	6 µs	500
ATOMS	QUBITS PER QPU	PROGRAM LENGTH	SHOTS PER JOB
defect-free atoms in one register, Pasqal's record (April 2026)	on today's Orion Beta and Orion Gamma machines	the longest pulse sequence the QPUs accept	the maximum repetitions in a single QPU job

Contents

Start here

How to use this guide · Pasqal in sixty seconds · The learning map

Part 1 · Quantum basics

Bits and qubits · Superposition · Measurement and shots · The Bloch sphere · Entanglement · Interference · Hamiltonians

Part 2 · How a neutral-atom QPU works

Why atoms · Inside the machine · Loading and sorting · Qubit encodings · Driving with light · The Rydberg blockade · The Hamiltonian · Readout · One shot · Noise

Part 3 · Ways to compute

Analog vs digital · Quantum simulation · The adiabatic algorithm · Maximum independent set · QUBO · Variational loops · Graph machine learning · Classical emulation

Part 4 · The platform map

Three layers · Which tool to use · docs.pasqal.com · What changed recently

Part 5 · Pulser, hands on

Install · Five objects · Units · Registers · Devices · Channels · Pulses · Sequences · First program · AFM state · Parameters · Emulation · Noise · Pulser Studio · Tutorials

Part 6 · Emulators and Pasqal Cloud

Choosing an emulator · emu-mps · The QPUs · Projects, batches and jobs · Batch life cycle · The SDK · Open batches · Credits · Partner clouds · Old vs new API · Pre-flight checklist

Part 7 · Higher-level tools

QoolQit · MIS solver · QUBO solver · QEK · Neutral Atom Toolkit · Qadence and others

Part 8 · Pasqal, the company

History · Deployments · Roadmap · Use cases · Comparison with other qubits

Part 9 · Keep going

Six-week plan · Cheat sheet · Glossary · Resources

The online version of this guide adds three interactive labs and a searchable glossary.

START HERE

How to use this guide

Pasqal builds quantum computers out of individual atoms held in place by laser light. To use its platform well you need two kinds of knowledge: a little quantum physics, and the software tools that turn your ideas into laser pulses. This guide teaches both, in that order.

0.1 Who this is for and how it is laid out

The guide assumes you can read basic Python and remember a little school maths. Everything else is explained as it comes up. Each part builds on the one before it, so the first time through, read in order.

Parts 1–3 · Concepts Qubits, measurement and entanglement; how atoms, lasers and the Rydberg blockade make a quantum computer; and the kinds of problems it solves.	**Part 4 · The map** How Pasqal's software and hardware fit together, and which tool to pick for which job.
Parts 5–7 · Hands on Pulser programs, emulators, Pasqal Cloud, and the higher-level libraries for optimisation and machine learning.	**Parts 8–9 · Context and practice** The company, its machines and roadmap, then a six-week study plan, a cheat sheet, a searchable glossary and links.

Look out for these boxes as you read:

KEY IDEA

The one thing to remember from a section.

EVERYDAY PICTURE

An analogy that gives you intuition. Analogies are never perfect; treat them as a first handhold.

IN PULSER

Where the concept shows up in Pasqal's software, with the exact class or function name.

WATCH OUT

A common mistake, or something that changed recently and trips people up.

There are also three small interactive labs (green borders) where you can move sliders and watch the physics respond, and a short **Check yourself** quiz at the end of most parts.

0.2 Pasqal in sixty seconds

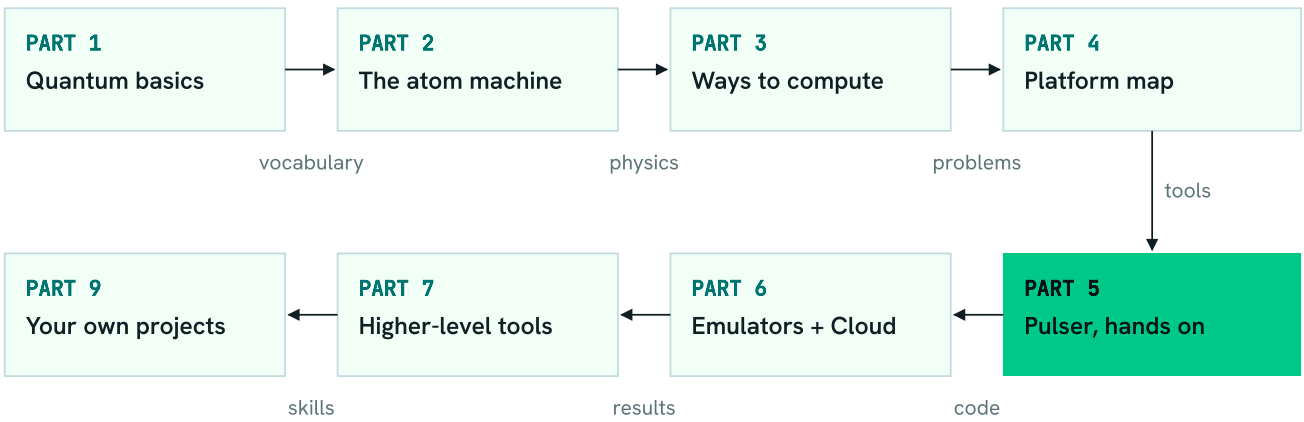
Pasqal is a French quantum computing company founded in 2019 as a spin-off of the Institut d'Optique near Paris. Its machines use **neutral atoms**: single rubidium atoms, cooled almost to absolute zero and held in a pattern by tightly focused laser beams called **optical tweezers**. Each atom is a **qubit**. Other lasers then push the atoms into very large, "puffed-up" states called **Rydberg states**, in which neighbouring atoms interact strongly. That interaction is what lets the atoms compute together.

Today's Pasqal machines (the **Orion** family) mostly run in **analog mode**: instead of a list of logic gates, you choose where the atoms sit and how the laser's strength and frequency change over a few microseconds. The atoms then evolve naturally, and you measure the result as a string of 0s and 1s. You repeat this hundreds of times and study the statistics.

To program these machines you use Pasqal's open-source Python library **Pulser**, or higher-level libraries built on top of it (**QoolQit**, and ready-made solvers for graph and optimisation problems). You test programs on **emulators** (classical software that imitates the QPU), then submit them through **Pasqal Cloud** or partner clouds such as Google Cloud, Microsoft Azure Quantum, OVHcloud and Scaleway.

0.3 The learning map

Here is the path you will follow. Each box is a part of the guide; the arrows show what each one gives you for the next.



The route through the guide. Part 8 (the company, its machines and roadmap) is background you can read at any point.

PART 1 · QUANTUM BASICS

The handful of quantum ideas you actually need

Quantum computing has a reputation for being mysterious. For using Pasqal's platform, you need about seven ideas, and each one has a concrete meaning in the lab. This part explains them with as little maths as possible, and shows where each one appears on real atoms.

YOU WILL LEARN

- What a qubit is, and how it differs from a bit
- Why results come as many "shots" and a histogram
- What entanglement and interference are
- What a Hamiltonian is, and why Pasqal talks about them constantly

TIME

- About 45 minutes
- School algebra is enough

1.1 Bits and qubits

A classical **bit** is always exactly 0 or 1, like a light switch. A **qubit** is a physical system with two distinct states that can also exist in a combination of both. On Pasqal's machines, a qubit is a single atom, and the two states are two of its energy levels: usually the **ground state**, written |g⟩, and a highly excited **Rydberg state**, written |r⟩. Pulser's own definition: when exactly two of an atom's energy levels are used to store information, that atom is a qubit. (Use three or more levels and it becomes a "qudit".)

The strange-looking brackets, |g⟩ and |r⟩, are just labels for states. Physicists call this notation a "ket". You will also see |0⟩ and |1⟩, which mean the same two states once they are used as computer bits.

THE STATE OF ONE QUBIT

$$|\psi\rangle = \alpha\,|g\rangle + \beta\,|r\rangle \qquad \text{with} \quad |\alpha|^2 + |\beta|^2 = 1$$

ψ "psi", the name of the state

α, β amplitudes: numbers that describe the blend

|α|² probability of measuring g (reads as 0)

|β|² probability of measuring r (reads as 1)

1.2 Superposition: amplitudes and probabilities

When both α and β are non-zero, the qubit is in a **superposition**. This does not mean "it is secretly 0 or 1 and we don't know which". It genuinely holds both possibilities at once, and the evidence is **interference** (section 1.6), which a hidden coin could never produce.

Amplitudes are richer than probabilities. Each one has a size and a **phase** (a direction, like the hand of a clock). Probability keeps only the size ($|\alpha|^2$) and throws the phase away. But the phase is what makes quantum effects possible, so the laser pulses you will write in Pulser control phase as carefully as they control probability.

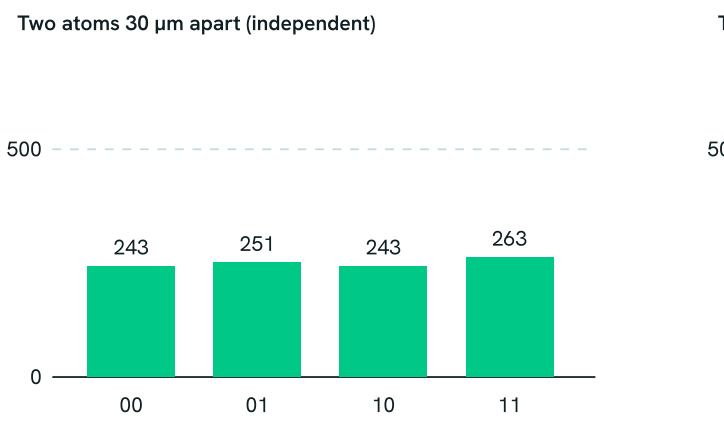
EVERYDAY PICTURE

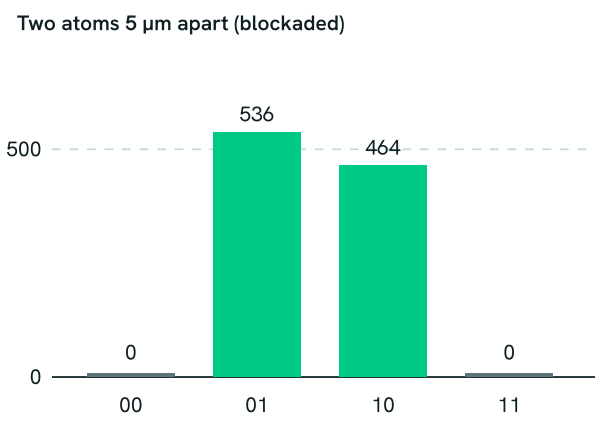
A spinning coin is neither heads nor tails while it spins; it carries a "recipe" for how likely each side is when it lands. The amplitude is the recipe, and the probability is what you see when it lands. The analogy breaks down in one important way: real amplitudes can cancel each other out, like waves.

1.3 Measurement, shots and bitstrings

Reading a qubit forces it to choose: you get $|g\rangle$ with probability $|\alpha|^2$ or $|r\rangle$ with probability $|\beta|^2$, and the superposition is gone. On Pasqal hardware measurement is **destructive** in a very literal sense. The atoms are photographed, the Rydberg atoms are lost, and that run is over.

So one run of a program gives you one **bitstring**: one 0 or 1 per atom, in register order. To learn the probabilities, you repeat the identical program many times. Each repetition is called a **shot** (Pasqal also says **run**). Results arrive as a count of how often each bitstring appeared, which you can draw as a histogram.





Counts out of 1,000 shots, simulated with Pulser 1.9.1 (QutipBackendV2).

The same laser, two outcomes. Left: each distant atom gets a half flip (area π/2) and behaves like a fair coin, so all four bitstrings appear about equally. Right: the atoms are close enough to block each other (Part 2), so the pair can hold only one excitation: you get 01 or 10, never 11. You will reproduce both in Part 5.

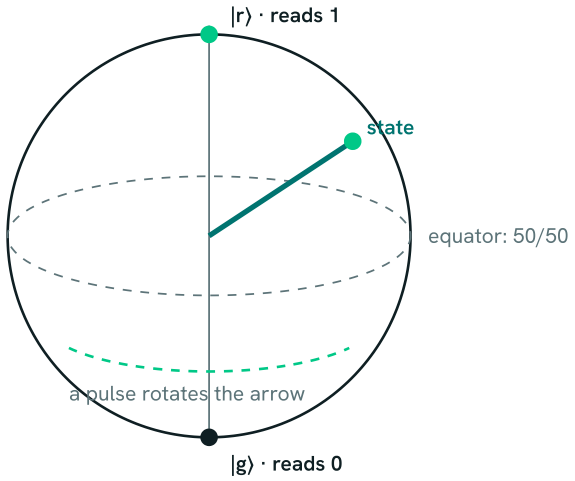
KEY IDEA

A quantum program's output is a **probability distribution**, estimated from shots. More shots give a sharper estimate: the statistical error shrinks roughly like $1/\sqrt{(\text{shots})}$. Pasqal's QPUs accept up to 500 shots per job, and each shot takes seconds, so choosing the shot count is a real budgeting decision.

1.4 The Bloch sphere: a picture of one qubit

Every possible state of a single qubit can be drawn as an arrow from the centre of a sphere to a point on its surface. This is the **Bloch sphere**. The north pole is $|r\rangle$ (reads as 1), the south pole is $|g\rangle$ (reads as 0), and every point on the equator is a 50/50

superposition; the points differ only in their phase. The height of the arrow sets the probabilities. Its direction around the equator is the phase.



What the laser does

Amplitude Ω	how fast the arrow turns
Phase φ	which horizontal axis it turns around
Detuning δ	tilts the axis up or down
Pulse area	total turning angle
π	pole to pole: a full flip
π/2	pole to equator: 50/50

One qubit as an arrow. Laser pulses turn the arrow; measurement asks "northern or southern hemisphere?" and answers with probabilities set by the arrow's height. In Pulser's convention the rotation axis is $(\Omega \cos\varphi, -\Omega \sin\varphi, -\delta)$, and the arrow turns at the rate $\sqrt{(\Omega^2 + \delta^2)}$.

1.5 Entanglement

Two or more qubits are **entangled** when their joint state cannot be described as "this atom's state" plus "that atom's state". The pair has a definite shared state even though neither atom has one of its own. Measuring one atom then tells you something about the other.

The right-hand histogram above shows this. After that pulse the two close atoms are in the state $(|g\,r\rangle + |r\,g\rangle)/\sqrt{2}$: "one of us is excited, but neither of us knows which". Each atom alone looks like a fair coin, yet the two results are perfectly anti-correlated: whenever one reads 1, the other reads 0. On Pasqal machines, entanglement comes from the strong interaction between Rydberg atoms, which Part 2 explains.

EVERYDAY PICTURE

Two coins sealed in a box, guaranteed to land opposite. Neither coin has an answer until you open the box, but the pair always does. (Real entanglement is stronger than any such hidden arrangement, which has been proven experimentally. Alain Aspect, one of the scientists connected to Pasqal's founding, shared the 2022 Nobel Prize in Physics for those experiments.)

1.6 Interference, and why quantum computers might help

Amplitudes add up like waves before they are squared into probabilities. Two contributions with opposite phase cancel; two with the same phase reinforce. Quantum algorithms are designed so that the amplitudes of wrong answers cancel and those of right answers build up. That is **interference**, and it is where any quantum speed-up comes from.

The reason quantum states are hard to imitate is size. A register of N qubits needs 2^N amplitudes to describe completely. Storing them on an ordinary computer takes 16 bytes each:

QUBITS	AMPLITUDES	MEMORY TO STORE THEM	COMPARABLE TO
10	1,024	16 KB	a tiny text file
20	≈1 million	16 MB	a phone photo
30	≈1 billion	16 GB	a laptop's memory
40	≈1 trillion	16 TB	a large server
50	$\approx 10^{15}$	16 PB	a top supercomputer
100	$\approx 10^{30}$	impossible	one of Pasqal's Orion Beta QPUs

WATCH OUT · A HUGE STATE SPACE IS NOT ENOUGH

Having 2^N amplitudes does not mean you can read them all. Each shot returns one bitstring. A quantum computer only wins when the program uses interference to concentrate probability on useful answers. And "quantum advantage" is always about a specific problem compared with the best classical method, which keeps improving.

1.7 Hamiltonians: the rulebook for how states change

You will meet the word **Hamiltonian** (written H) constantly in Pasqal's documentation. A Hamiltonian is the system's energy function. It lists every influence acting on the atoms (the lasers, the interactions between atoms) and assigns an energy to every possible arrangement. It is also the rule that says how the quantum state changes over time, through the **Schrödinger equation**:

SCHRÖDINGER EQUATION (AS WRITTEN IN THE PULSER DOCS)

$$i\frac{d\,|\,\Psi(t)\rangle}{dt} = \frac{H(t)}{\hbar}\,|\,\Psi(t)\rangle$$

`|Ψ(t)⟩` the state of all the atoms at time t

`H(t)` the Hamiltonian; it changes as your pulses change

`ħ` reduced Planck constant, a fixed number

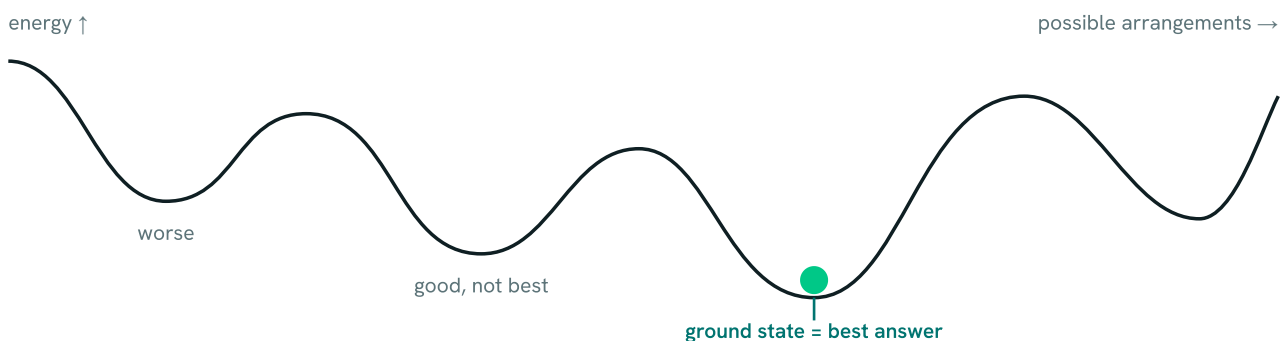
`i` the imaginary unit; it is why phases rotate

You never solve this equation on the QPU yourself. "Evolving under a Hamiltonian" means that you **set up** H, by choosing where the atoms sit and what the laser does, and then **let physics run**. The atoms do the calculation simply by existing. An emulator does the same calculation slowly, on a classical computer.

Ground states and energy landscapes

Because H gives every arrangement an energy, you can picture an **energy landscape**: hills for costly arrangements, valleys for cheap ones. The lowest point is the **ground**

state. Most of Pasqal's optimisation methods rest on one sentence: *encode your problem so that its best answer is the ground state, then guide the atoms into it.*



Optimisation as finding the lowest valley. Classical search can get stuck in shallow valleys. Pasqal's analog algorithms try to carry the whole quantum state smoothly into the deepest one (Part 3, the adiabatic algorithm).

EVERYDAY PICTURE

Pour water onto a crumpled sheet of paper. You don't compute where it will go; it finds the dips by itself. Building the right Hamiltonian is crumpling the paper so that the deepest dip is your answer.

CHECK YOURSELF · PART 1 — tap a question to reveal the answer

Q A qubit has $\alpha = \beta = 1/\sqrt{2}$. What are the chances of reading 0 and 1?

$|\alpha|^2 = |\beta|^2 = 1/2$, so 50% each. That state sits on the equator of the Bloch sphere.

Q Why can't you learn a qubit's amplitudes from a single shot?

A shot returns one bit and destroys the superposition. You estimate probabilities by repeating the program many times and counting.

Q In the right-hand histogram, each atom alone reads 1 about half the time. Why is the pair entangled rather than two independent coins?

Independent coins would sometimes both read 1 (and sometimes both 0). Here 11 and 00 never happen: the results are perfectly anti-correlated, which a product of two separate states cannot produce.

Q In one sentence, what is a Hamiltonian?

The energy function of the system, which also dictates how its quantum state evolves in time.

PART 2 · HOW A NEUTRAL-ATOM QPU WORKS

Atoms, tweezers, lasers and the blockade

This part opens the box. You will follow a single shot from a cloud of rubidium gas to a camera image, and learn the three controls (amplitude, detuning, phase) and the one interaction (the Rydberg blockade) that every Pasqal program is built on.

YOU WILL LEARN

- The parts of the machine and what each one does
- How atoms are loaded, sorted, driven and read out
- The Rydberg blockade and the blockade radius
- The Hamiltonian Pasqal's QPUs implement, term by term
- Where noise comes from

TIME

- About 75 minutes
- Two interactive labs

2.1 Why build a computer out of atoms?

Pasqal's machines use single atoms of **rubidium-87**, a soft metal whose atoms are easy to cool and trap with standard lasers and can be excited to very high energy levels. The atoms are **neutral** (they carry no electric charge), which is what distinguishes this approach from trapped-ion computers. Pasqal gives three main reasons for the choice:

Identical by nature

Every rubidium atom is a perfect copy of every other. There is no manufacturing variation to calibrate away, unlike qubits printed on a chip.

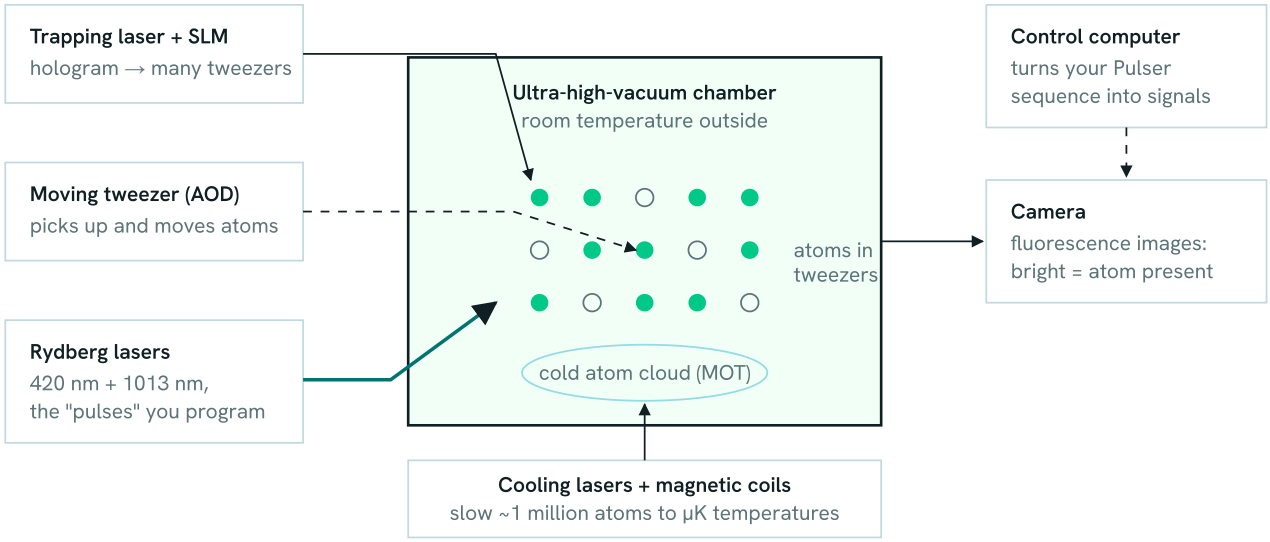
No giant fridge

The atoms are cooled to millionths of a degree above absolute zero, but only the atoms. The vacuum chamber and the room stay at room temperature; an Orion Beta system draws about 10 kW.

Any shape you like

Lasers place atoms in almost any pattern in 1D or 2D (3D in research). Changing the layout is a software change, not a new chip.

2.2 Inside the machine



The main parts of a neutral-atom QPU (simplified). Only the Rydberg-laser pulses are what you program in Pulser; everything else is the machine preparing and reading out the atoms for you.

Magneto-optical trap (MOT)

Laser beams and a magnetic field slow a cloud of about a million atoms (roughly 1 mm^3) until it is a few millionths of a degree above absolute zero. Temperature *is* motion, so slowing atoms cools them.

Optical tweezers

Tightly focused laser spots about 1 μm wide. An atom is pulled into the brightest point of a spot and held there. One tweezer holds at most one atom, so one tweezer means one qubit.

SLM (spatial light modulator)

A programmable hologram that splits one laser into hundreds of fixed tweezer spots in any pattern. It creates the trap **layout**.

AOD (acousto-optic deflector)

A crystal driven by sound waves that steers a laser beam. It makes a **movable** tweezer, used to carry atoms around when sorting them.

Rydberg excitation lasers

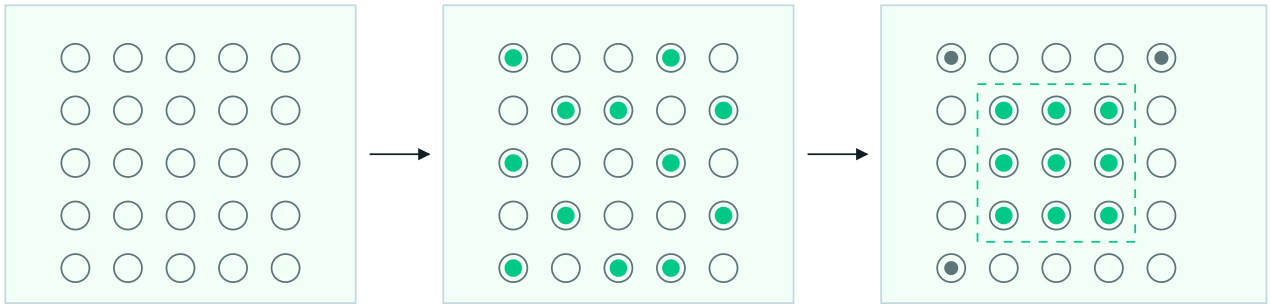
No single laser can lift rubidium from the ground state to a Rydberg level, so Pasqal's Fresnel machines use two photons: 420 nm (violet) then 1013 nm (infrared), tuned to act as one transition.

Camera

Atoms lit by resonant light glow (fluoresce). A sensitive camera records which trap sites are bright, which is how atoms are counted and results are read.

2.3 Loading and sorting atoms

Filling the tweezers is a game of chance: each one captures an atom only about **50%** of the time. A fresh array therefore comes up roughly half full, in a different random pattern on every shot. To get the exact register you asked for, the machine photographs the array, then a moving tweezer carries atoms from spare traps into the empty sites of your pattern. This **rearrangement** succeeds more than 99% of the time, and each move takes under a millisecond.

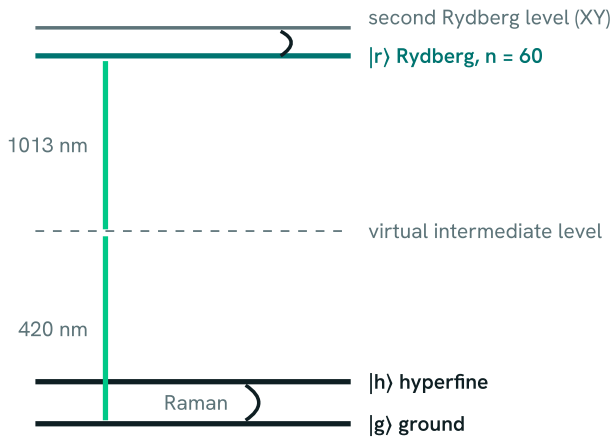


1 · trap layout (25 traps) 2 · random loading (12 of 25) 3 · sorted into your register

Why layouts have spare traps. Because loading is random, the machine needs more traps than atoms. Pasqal's QPUs allow a register to use at most 45% of a layout's traps (up to 200 traps in a layout), so there are always enough atoms to sort into place. The hero animation at the top of this page shows the same process.

2.4 Turning an atom into a qubit

An atom has many energy levels. A qubit uses two of them. Pasqal's software knows three choices, called **bases**, and each is driven by a different kind of light:



Three ways to make a qubit

`ground-rydberg`
|g⟩ ↔ |r⟩ · Rydberg laser
strongly interacting, analog workhorse;
the only basis on today's cloud QPUs

`digital`
|g⟩ ↔ |h⟩ · Raman lasers
long-lived memory for gate-based mode

XY
two Rydberg levels · microwaves
longer-range "exchange" interaction

Energy levels used as qubits (not to scale: the Rydberg level is vastly further up than drawn). The Rydberg state is short-lived but makes atoms interact strongly; the hyperfine levels last almost indefinitely but barely interact, so gate-based schemes park information there and visit |r⟩ briefly to entangle.

2.5 Driving atoms with light: Ω, δ and φ

A laser pulse acting on the |g⟩ ↔ |r⟩ transition is described by exactly three numbers that can change over time. These are the only knobs you have, and every Pulser `Pulse` sets all three.

Amplitude Ω (Rabi frequency)	Detuning δ	Phase φ
How strongly the laser drives the atom, which sets how fast it swings between |g⟩ and |r⟩. In rad/µs. Pasqal's QPUs allow up to 2 MHz, which is 12.57 rad/µs.	How far the laser frequency is from the atom's exact transition frequency. δ = 0 is "on resonance". In energy terms, δ is the reward (positive) or cost (negative) of being excited.	The laser's starting phase. It does not change the speed, only the axis of rotation on the Bloch sphere. Constant within a single pulse.

With the laser on, the atom oscillates back and forth between |g⟩ and |r⟩. These are **Rabi oscillations**. For a constant pulse, the probability of finding the atom in |r⟩ after time t is:

RABI OSCILLATION OF ONE ATOM

$$P_r(t) = \frac{\Omega^2}{\Omega^2 + \delta^2} \sin^2\left(\frac{\sqrt{\Omega^2 + \delta^2}\, t}{2}\right)$$

on resonance (δ = 0) the atom swings fully between 0% and 100%

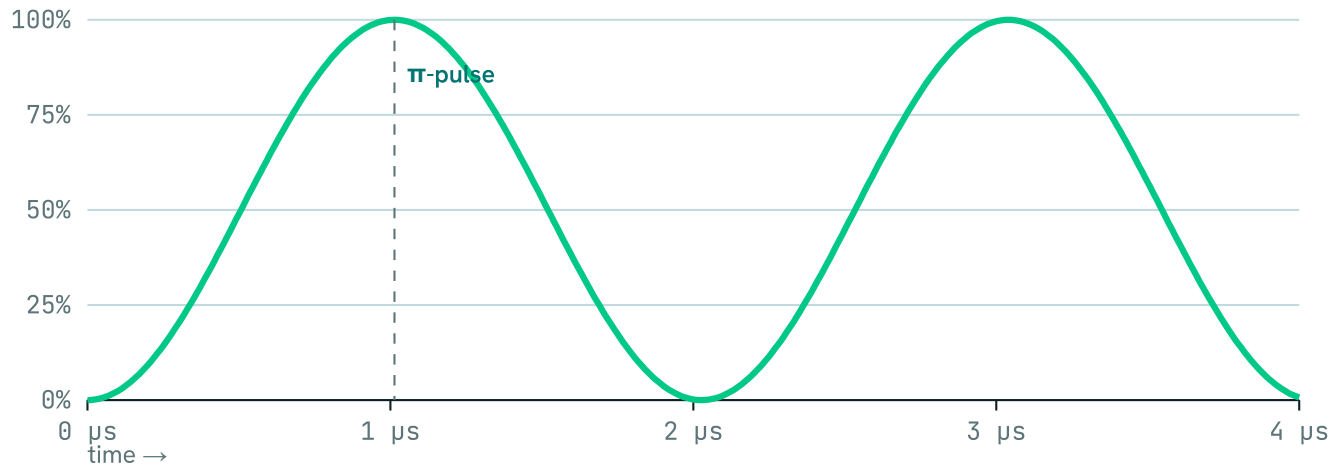
detuned (δ ≠ 0) it swings faster but never fully reaches |r⟩

pulse area ∫Ω dt; area π = full flip (π-pulse), π/2 = 50/50

LAB 1 · RABI OSCILLATIONS

Move the sliders; the curve shows P(atom in |r⟩) over 4 µs

Interactive lab: open the online version of this guide to move the sliders. Shown here in its starting state.



Highest P(r) reached	π-pulse duration (δ = 0)	Oscillation period
100 %	1013 ns	2.03 µs

Try it: with δ = 0, doubling Ω halves the π-pulse time. Add detuning and the oscillation speeds up but no longer reaches 100%. Detuning is how you make the atoms "reluctant" or "eager" to be excited, which matters a great deal in Part 3.

Global and local addressing

A **global** laser illuminates the whole array, so every atom receives the same Ω, δ and φ. A **local** beam targets chosen atoms. Pasqal's current cloud QPUs offer only a global Rydberg laser. That sounds limiting, but it is the normal mode for analog computing: because every atom gets the same pulse, the individual behaviour comes from **where the atoms are placed**, through the interaction described next.

> EVERYDAY PICTURE
>
> A global laser is one loudspeaker for the whole room; a local laser whispers to one person. With only the loudspeaker, your "program" is largely the seating plan.

2.6 Rydberg atoms and the blockade

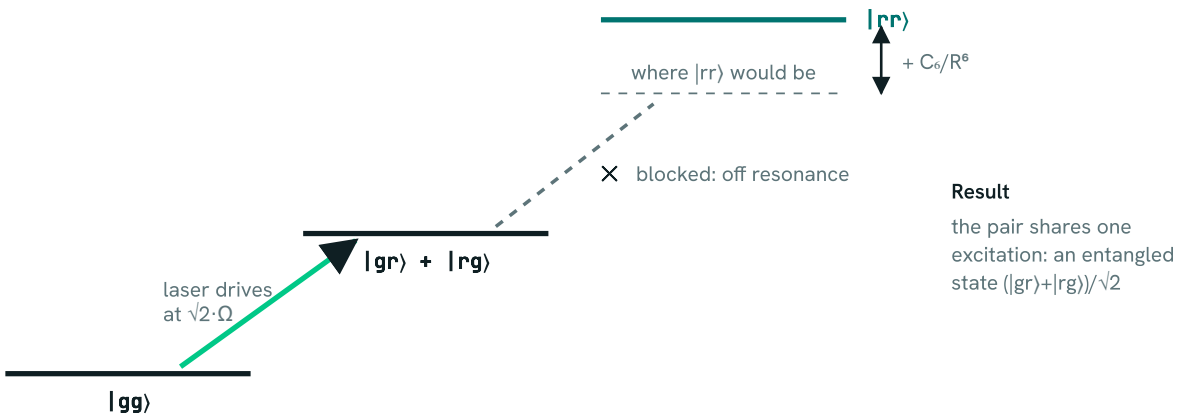
In a **Rydberg state**, the atom's outermost electron is pushed into a very high orbit (principal quantum number n = 60 on Pasqal's QPUs). The orbit's radius grows like n^2, so the atom becomes thousands of times larger than normal and extremely sensitive to its neighbours. Two Rydberg atoms a distance R apart repel each other with an energy that falls off very steeply:

VAN DER WAALS INTERACTION BETWEEN TWO RYDBERG ATOMS

$$U(R) = \frac{C_6}{R^6}$$

C_6 a constant set by the Rydberg level; 865,723 rad·μm⁶/μs for n = 60	**R** distance between the atoms, in μm	**$1/R^6$** double the distance and the interaction drops 64×

Now the key effect. Suppose two atoms sit close together and a laser tries to excite both. Exciting the first is fine. But exciting the second as well would cost the extra energy U(R), which shifts the "both excited" state |rr⟩ so far off resonance that the laser simply cannot reach it. The pair can hold **at most one** excitation. This is the **Rydberg blockade**, first demonstrated between two individual atoms in 2009 by Antoine Browaeys's group, whose members later co-founded Pasqal.



Why the blockade happens. When the interaction C_6/R^6 is much larger than the laser's strength, the doubly excited state is out of reach. The two atoms then oscillate together between "none excited" and "one excited, shared", and do so $\sqrt{2}$ times faster than a single atom. More generally, N atoms inside one blockade region share a single excitation and oscillate $\sqrt{N}$ times faster.

The blockade switches on when the interaction beats the laser drive: C_6/R^6 greater than Ω. Setting the two equal defines the **blockade radius**:

BLOCKADE RADIUS

$$R_b = \left(\frac{C_6}{\Omega}\right)^{1/6}$$

Ω = 12.57 (device maximum) → $R_b \approx 6.4$ µm

Ω = 2π → $R_b \approx 7.2$ µm

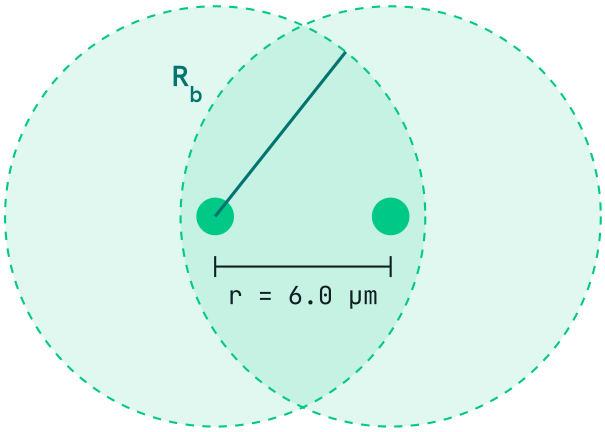
Ω = 1 → $R_b \approx 9.8$ µm

In Pulser `device.rydberg_blockade_radius(Ω)`

Atoms closer than R_b block each other; atoms further apart act almost independently. It is a soft crossover, not a hard wall, and the approximation is least accurate when R is close to R_b. Notice two practical facts. The blockade radius at full power (about 6.4 µm) is just above the QPU's 5 µm minimum spacing, so nearest neighbours block and next-nearest usually don't. And you can change R_b without moving any atom, simply by changing Ω.

LAB 2 · THE BLOCKADE RADIUS — Change the spacing and the laser strength

Interactive lab: open the online version of this guide to move the sliders. Shown here in its starting state.



1 μm = 14 px · dashed circle = blockade radius

Blockade radius R_b	Interaction C_6/R^6	Interaction ÷ drive
7.18 μm	18.56 rad/μs	2.95×

Blockaded. The atoms sit inside each other's blockade radius, so the laser cannot put both into the Rydberg state. The pair shares one excitation and oscillates √2 times faster.

KEY IDEA

The blockade is how neutral atoms compute. It creates entanglement, it enforces "these two can't both be 1" constraints for free, and its range is set by geometry and laser power. Almost every algorithm in Part 3 is a clever use of it.

2.7 The Hamiltonian, term by term

Putting the pieces together gives the Hamiltonian that Pasqal's analog QPUs implement. It has three terms: one for the laser drive, one for the detuning, and one for the interactions between every pair of atoms.

RYDBERG (ISING-TYPE) HAMILTONIAN, AS WRITTEN IN PULSER

$$H(t) = \sum_i \frac{\Omega(t)}{2}\left(\cos\varphi\,\sigma_i^x - \sin\varphi\,\sigma_i^y\right) - \sum_i \delta(t) n_i + \sum_{i<j} \frac{C_6}{R_{ij}^6} n_i n_j$$

- σ^x, σ^y operators that flip an atom between |g⟩ and |r⟩
- n_i "is atom i excited?" (1 if in |r⟩, 0 if not)
- R_{ij} distance between atoms i and j
- Σ sum over all atoms (or all pairs)
- **units** written with ħ = 1, so energies are in rad/μs

1 · Drive: the mixer	2 · Detuning: the reward	3 · Interaction: the penalty
Ω flips atoms back and forth and creates superpositions, letting the system explore many arrangements at once.	−δ·n: each excited atom changes the energy by −δ. Positive δ makes excitations cheap ("get as many 1s as you can"); negative δ makes them costly.	C_6/R^6 for every pair that is *both* excited. Always positive, always on, and set purely by where the atoms are.

EVERYDAY PICTURE · SEATING A WEDDING

The drive is guests milling around. The detuning is "you score a point for every guest who sits down". The interaction is "these two guests must not both sit; they're feuding". You want the most points without seating a feuding pair, and you describe the feuds purely by **where you put the chairs**. That is exactly how Pasqal encodes optimisation problems.

WATCH OUT · SIGN CONVENTIONS

The minus sign in front of sin φ comes from Pulser's internal ordering of states (|r⟩ listed first). If you relabel |g⟩ as 0 and |r⟩ as 1 in the usual textbook way, that sign becomes plus. Pulser's *Conventions* page spells out both. Also note that the interaction term never turns off, even during a delay with the laser off: atoms always "feel" each other whenever more than one is excited.

Two variants also appear in the docs. In **XY mode** (two Rydberg levels driven by microwaves), the interaction lets atoms swap an excitation, with strength C_3/R^3 that reaches further than C_6/R^6. With a **detuning map** (DMM), an extra term $-\Sigma\ \varepsilon_k\ \delta_{DMM}(t)\ n_k$ gives each atom its own weight ε_k between 0 and 1.

2.8 Reading out the answer

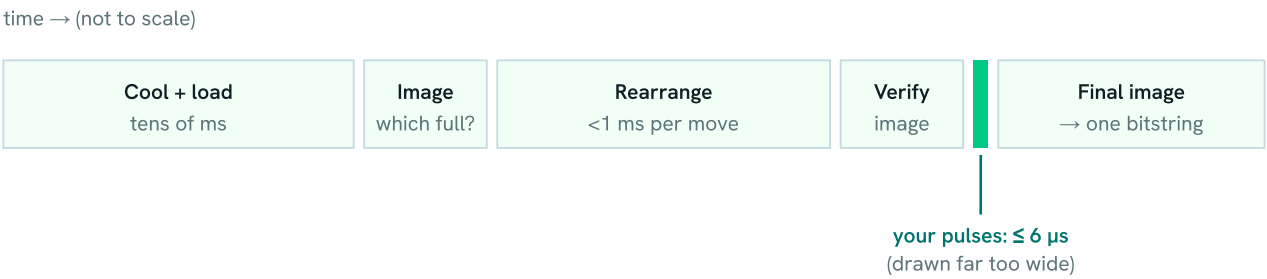
At the end of a sequence the machine takes another fluorescence image. Here is the clever part: the tweezers hold ground-state atoms but **repel** Rydberg atoms, which therefore escape. So in the final image a **bright** site means the atom was in |g⟩ (read as **0**), and a **dark** site means it was in |r⟩ and left (read as **1**). Detection efficiency in this kind of setup exceeds 98.6%.

EVERYDAY PICTURE

A row of candles. Your pulse blows across them; the candles that went out are the atoms you excited. You read the answer off which ones are dark.

Because the measurement removes atoms, you get one bitstring per shot, you cannot peek at the state mid-sequence, and the whole register must be rebuilt for the next shot.

2.9 One shot, from start to finish



Raw lab cycle ≈ 200 ms (about 5 shots per second). Pasqal Cloud estimates cost at 4 s per shot (0.25 Hz), counting only successful runs.

Where the time goes. The quantum computation itself is a few microseconds; almost all of the cycle is preparing and checking atoms. This is why shots are precious on neutral-atom machines, and why a 500-shot job takes minutes rather than milliseconds.

Different sources quote different repetition rates because they measure different things. A research setup's raw cycle is about 200 ms (around 5 Hz). Pasqal's Orion Beta specification gives a minimum *effective* rate of 0.25 Hz, counting only successful computations, and notes that it depends on the number of qubits. Pasqal Cloud uses the same 4 seconds per shot as a deliberately cautious cost estimate.

2.10 Noise: what goes wrong, and why it matters

Real atoms and real lasers are imperfect. Pulser's `NoiseModel` is effectively Pasqal's published list of what can go wrong, so learning the noise sources also teaches you the emulator's settings (Part 5).

NOISE SOURCE	WHAT HAPPENS	EVERYDAY PICTURE
State preparation error	An atom is missing at the start (failed loading or lost while sorting)	An empty seat you thought was taken
False positive	A ground-state atom is lost for another reason and its dark site reads as 1	Someone stepped out for coffee and was counted as "excited"
False negative	A Rydberg atom decays back and is recaptured, so it reads as 0	Someone sat back down just before the photo
Dephasing	The phase between $\|g\rangle$ and $\|r\rangle$ drifts randomly	A choir in tune but drifting out of time
Laser amplitude	The beam is brighter in the middle (finite waist) and fluctuates from shot to shot	A spotlight that is dimmer at the edges of the stage
Laser detuning	The laser frequency jitters	A radio drifting slightly off-station
Doppler shift	Moving atoms see a slightly shifted laser frequency	A siren's pitch changing as it moves
Position jitter	Atoms wobble in their traps, and C_6/R^6 magnifies tiny changes in R	Chairs that slide a little each time
Rydberg decay	The excited state has a finite lifetime and decays back	Standing on a chair is tiring; eventually you sit down

This is one reason sequences are capped at 6 µs: the longer the program, the more these effects accumulate. Noisy emulation tells you in advance whether a job is worth its queue time, which imperfection is hurting you, and how to design pulses that tolerate it.

CHECK YOURSELF · PART 2 — tap a question to reveal the answer

Q Why can a register use at most 45% of a layout's traps?

Tweezers load randomly, about half the time each. Spare traps guarantee enough atoms are available to sort into your requested pattern.

Q You double the distance between two Rydberg atoms. What happens to their interaction?

It drops by $2^6 = 64$ times, because it scales as $1/R^6$.

Q How can you shrink the blockade radius without moving any atoms?

Increase the laser amplitude Ω. $R_b = (C_6/\Omega)^{\wedge}(1/6)$ gets smaller as Ω grows.

Q In the final image, a site is dark. What bit does that atom give, and why?

1. The atom was in the Rydberg state, which the tweezer repels, so it escaped before the image.

Q In the Hamiltonian, which term encodes "these two atoms can't both be 1"?

The interaction term $C_6/R_{ij}^6 \cdot n_i\, n_j$. It costs a lot of energy only when both atoms are excited.

PART 3 · WAYS TO COMPUTE

From physics to problems worth solving

You have a register of atoms, a laser you can shape, and an interaction set by geometry. This part shows how that becomes computation: analog and digital modes, simulating materials, the adiabatic algorithm, independent sets and QUBOs, feedback loops, graph machine learning, and the limits of simulating all this on classical computers.

YOU WILL LEARN

- Analog vs digital vs digital-analog computing
- The adiabatic recipe behind most Pasqal applications
- Why maximum independent set is the "native" problem
- How QUBOs, variational loops and quantum kernels work

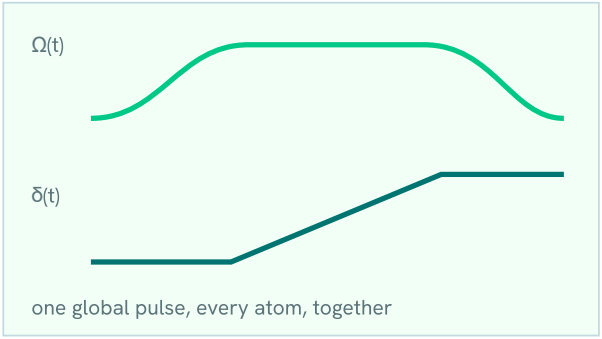
TIME

- About 60 minutes
- One interactive puzzle

3.1 Analog, digital and digital-analog

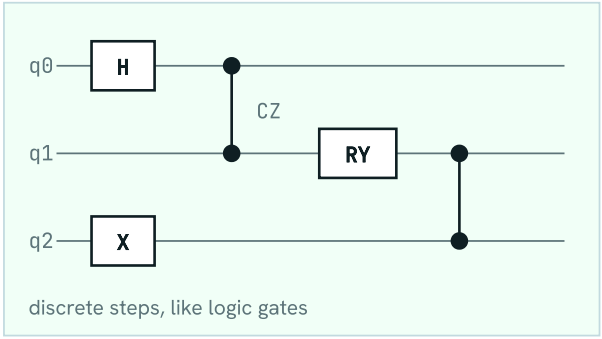
There are two broad ways to program a quantum computer, and neutral atoms can do both.

Analog shape the whole system's Hamiltonian over time



one global pulse, every atom, together

Digital a sequence of gates on chosen qubits



discrete steps, like logic gates

Two programming styles. Analog mode treats the register as one physical system to be steered; digital mode breaks a computation into gates. Today's Pasqal cloud QPUs run in analog mode.

Analog

No gates. You place atoms and shape Ω(t) and δ(t), and the whole register evolves under one Hamiltonian. Pasqal argues this suits today's noisy machines: one smooth analog step can replace a very long chain of error-prone gates.

Digital

Single-qubit gates come from Raman lasers on the hyperfine qubit (|g⟩, |h⟩). Two-qubit gates use the blockade. Because many atoms can share one blockade region, multi-qubit gates such as Toffoli can be native.

Digital-analog

A hybrid: digital gates on the hyperfine levels, combined with blocks of analog evolution that use the Rydberg level for entanglement. This was the focus of Pasqal's Qadence library (Part 7).

How the blockade makes a two-qubit gate

The classic controlled-Z (CZ) gate uses three pulses: a π-pulse on the control atom, a 2π-pulse on the target, then another π-pulse on the control. A 2π round trip from |g⟩ to |r⟩ and back leaves the target with a minus sign. But if the control was excited, the blockade stops the target's round trip, so no minus sign appears. That conditional minus sign is exactly a CZ gate. In research setups such gates take about a microsecond.

> EVERYDAY PICTURE
>
> The target tries to go out for a walk and come back, and gets a stamp on its passport when it does. If the control is standing in the doorway, the target can't leave, and the stamp is missing. The missing stamp is the gate.

3.2 Quantum simulation: atoms imitating materials

The Hamiltonian in Part 2 is, almost exactly, a textbook model of **magnetism** (the transverse-field Ising model). Each atom acts like a tiny magnet that can point up (excited) or down (ground). So the most direct use of a neutral-atom QPU is to **simulate magnetic materials** by building them atom by atom and watching what happens. This is the oldest and best-established use of the technology. Pasqal's own blog points to a 2021 *Nature* experiment by the Browaeys and Lahaye group with 196 atoms, and a 2026 preprint reports a one-to-one simulation of a frustrated magnet with 256 qubits.

A key concept here is the **quantum phase transition**. As you change δ/Ω, the lowest-energy arrangement of the atoms can suddenly reorganise: from "all in |g⟩" to a checkerboard where every second atom is excited (called **Z_2 order**), or, if the blockade covers two neighbours, to "every third atom excited" (**Z_3 order**). These transitions are driven by quantum effects, not by temperature, and they are exactly where classical simulation struggles most.

> EVERYDAY PICTURE
>
> Seat people with the rule "no two neighbours" and you get every other seat. Change the rule to "no one within two seats" and the pattern suddenly jumps to every third seat. The point where the pattern flips is the phase transition.

3.3 The quantum adiabatic algorithm

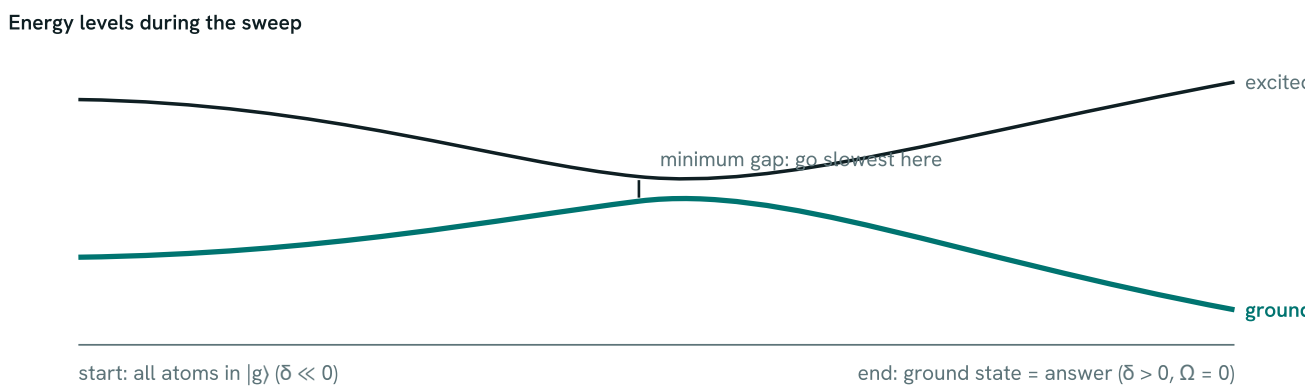
The **adiabatic theorem** says that if a system starts in its lowest-energy state and you change its Hamiltonian *slowly enough*, it stays in the lowest-energy state of the Hamiltonian at each moment. That gives a general recipe for optimisation, the **Quantum Adiabatic Algorithm (QAA)**:

1. **Start easy.**
   With strongly **negative** detuning, excitations are expensive, so the lowest-energy state is simply "every atom in |g⟩". That is exactly how the atoms start.

2 **Sweep slowly.**
Turn the laser on, then sweep δ from negative, through zero, to **positive**. Excitations become rewarded, but the blockade still forbids neighbours both being excited.

3 **End at the problem.**
Turn the laser off at the end. Now the lowest-energy state is the best arrangement of excitations allowed by your geometry: the answer.

4 **Measure many times.**
The most frequent bitstrings are the best candidates.



Why "slowly" matters. The system rides the lower curve. Where the two curves almost touch (the minimum gap), a fast sweep can kick it onto the upper curve and you lose the answer. Smaller gaps need slower sweeps; Pasqal's 6 µs program limit makes this a real trade-off against noise.

EVERYDAY PICTURE

Carrying a full bowl of soup across a room. Walk slowly and the surface stays flat; rush and it sloshes over. The narrowest doorway, where you must slow down most, is the minimum gap.

The antiferromagnet program in Part 5 is a QAA. Pulser's QUBO tutorial uses the same shape with δ swept from −5 to +5 rad/µs over 4 µs, and shows the average cost improving as the sweep gets longer.

3.4 Maximum independent set: the native problem

Draw a graph: dots (nodes) joined by lines (edges). An **independent set** is a group of nodes with no edge between any two of them. The **maximum independent set (MIS)** is the largest such group. Finding it is famously hard for large graphs (NP-hard), and it appears everywhere: scheduling jobs that can't overlap, placing antennas that mustn't interfere, choosing non-conflicting satellite observations.

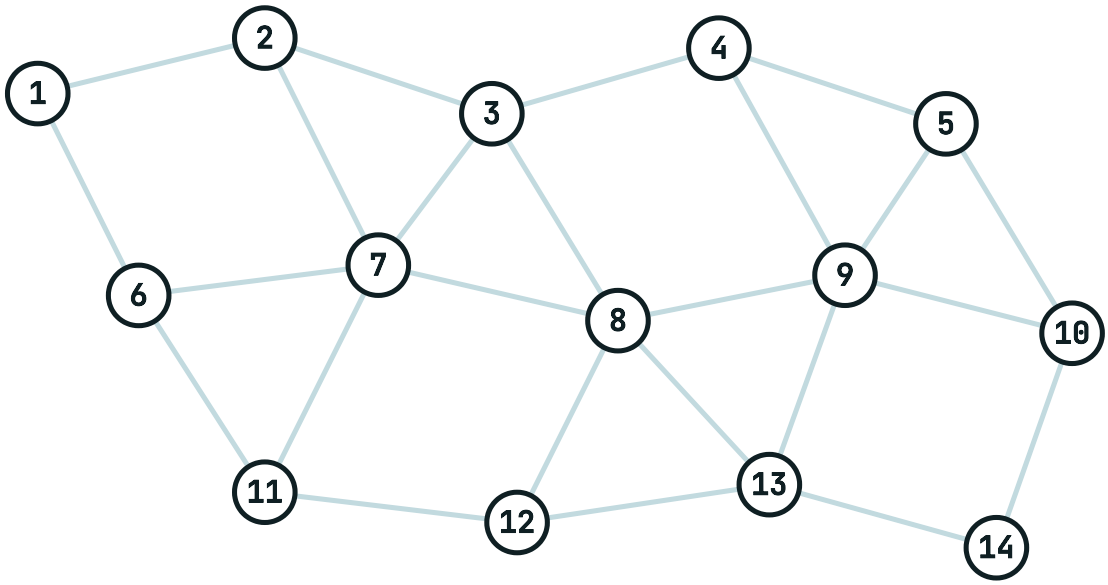
Now take a **unit-disk graph**: nodes are points on a plane, and two nodes are joined exactly when they are closer than some distance. Put one atom on each node and set the laser so the blockade radius equals that distance. Physics now enforces the rule for you: **two connected atoms cannot both be excited**. Every pattern of excitations the

atoms can reach is an independent set, and the detuning rewards having as many excitations as possible. So the ground state at the end of an adiabatic sweep is the maximum independent set. There is no encoding overhead: the hardware's own constraint is the problem's constraint.

LAB 3 · FIND THE MAXIMUM INDEPENDENT SET

Tap or click atoms to excite them. Linked atoms are inside each other's blockade radius.

Interactive lab: open the online version of this guide to move the sliders. Shown here in its starting state.



Excited / best possible	Blockade violations	Bitstring (atoms 1→14)
0 / 5	0	00000000000000

Valid so far: no two excited atoms are neighbours. Can you fit more? The best possible here is **5**.

On a real QPU you don't click; the adiabatic sweep finds high-quality sets on its own, and the bitstrings with the most 1s (and no violations) are your candidates. Pasqal's **MIS solver** library (Part 7) wraps all of this: give it a graph, get back independent sets.

Weighted problems (MWIS)

In the **maximum weighted independent set** problem, each node has a value and you maximise the total value rather than the count. Pasqal handles the weights with a detuning map (DMM): each atom gets an extra detuning according to its weight, so heavier nodes are more strongly rewarded for being excited. Pulser's MWIS tutorial walks through a four-node example whose solution is `0110`.

3.5 QUBO: binary optimisation in matrix form

Many business problems (portfolios, routes, schedules) can be written as a **QUBO**, a Quadratic Unconstrained Binary Optimisation. You have yes/no decisions $z_1 \ldots z_n$ (each 0 or 1) and a symmetric table of numbers Q. The goal is to choose the decisions that make this cost as small as possible:

QUBO COST

$$f(z) = \sum_{i,j} Q_{ij} z_i z_j \qquad z_i \in \{0, 1\}$$

Q_{ij} **(i ≠ j)** cost of choosing both i and j → set by atom distance, C_6/R_{ij}^6	Q_{ii} cost of choosing i alone → set by detuning	z_i **= 1** atom i excited

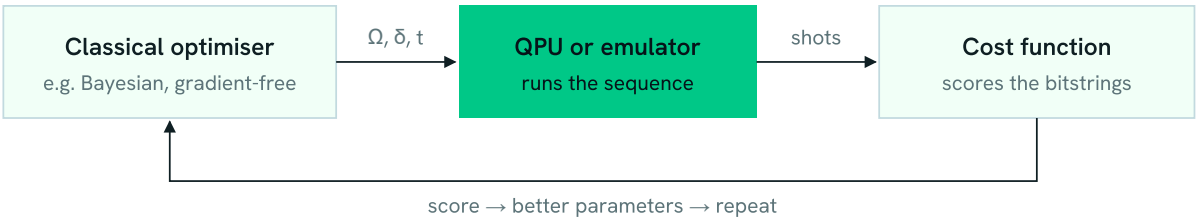
The mapping onto atoms is direct: the "both chosen" costs become interactions, which you control by **placing atoms at the right distances**; the single costs become the detuning. There are real limits, though. Interactions are always positive, so only QUBOs with positive off-diagonal entries embed directly. And because interaction strength is a fixed function of distance in 2D, an arbitrary matrix usually can't be reproduced exactly. Finding good positions is itself an optimisation problem; Pasqal's QUBO solver ships dedicated embedding methods (Greedy Layout, BLaDE) for it.

EVERYDAY PICTURE

You have a table of how much each pair of guests dislike each other, and only one tool: distance. Closer means more friction. Embedding a QUBO is arranging the chairs so the distances reproduce the dislike table as closely as possible, and sometimes no arrangement fits perfectly.

3.6 Variational loops: a classical computer in charge

Often you don't know the best pulse shape in advance. In a **variational** (hybrid quantum-classical) algorithm, a classical optimiser proposes pulse parameters, the QPU runs them and returns bitstrings, a cost function scores the result, and the optimiser proposes better parameters. **QAOA** (the Quantum Approximate Optimisation Algorithm) is the best-known example: it alternates between "problem" and "mixing" evolutions whose durations are tuned in the loop.



The hybrid loop. The quantum machine is an oven you can't see inside: you set the dials, taste the result, and adjust. On Pasqal Cloud, an "open batch" keeps the QPU reserved so each iteration doesn't rejoin the queue (Part 6).

Pulser's tutorial on Bayesian optimisation tunes the antiferromagnet schedule this way, adjusting Ω and δ at a few interpolation points to maximise a score of how checkerboard-like the result is.

3.7 Quantum machine learning on graphs

Graphs are a natural data type for neutral atoms, because a register *is* a graph. Pasqal's flagship machine-learning method, the **Quantum Evolution Kernel (QEK)**, uses that directly:

1. **Embed.**
   Turn each graph (for example a molecule: atoms as nodes, bonds as edges) into a register, with edges becoming blockade pairs.
2. **Evolve.**
   Apply the same fixed pulse to every graph's register.
3. **Sample.**
   Measure many shots. The bitstring distribution is a "fingerprint" of the graph's structure.
4. **Compare.**
   The similarity of two graphs is computed from how different their fingerprints are.
5. **Learn.**
   Feed the similarity table to an ordinary classical classifier (a support vector machine).

QUANTUM EVOLUTION KERNEL

$$K(G, G') = \exp\left(-\mu \, \mathrm{JS}(P_G, P_{G'})\right)$$

P_G bitstring distribution from graph G

JS Jensen–Shannon divergence: 0 when identical

μ a tuning knob (hyperparameter)

K = 1 graphs look identical to the QPU

Pasqal ran QEK on its QPU to predict chemical toxicity (on a public molecule dataset), and describes it as the first implementation of a quantum graph machine-learning algorithm on neutral-atom hardware.

EVERYDAY PICTURE

Play the same piece of music in differently shaped concert halls. Each hall's echo is unique. You can compare two halls by comparing their echoes, without ever seeing a blueprint.

3.8 Simulating atoms on classical computers

Emulators are central to the platform: you develop and test on them, and you compare QPU results against them. How far they can go depends on how they store the quantum state.

State vector (exact)

Store all 2^N amplitudes. Exact for any entanglement and geometry, but every extra atom doubles the memory and time. Practical up to about 25 atoms (EMU-SV), less with noise.

Matrix product state (approximate)

Store a chain of small tensors, one per atom, linked by "bonds". The **bond dimension** measures how much entanglement the chain can hold. Low entanglement: small bonds, cheap. High entanglement: bonds explode. Up to 80 atoms on EMU-MPS.

The catch is that entanglement peaks exactly at the quantum phase transitions that adiabatic algorithms must pass through. Two-dimensional registers make it worse, because a 2D grid must be "snaked" into a 1D chain, turning short-range neighbours into long-range chain links. So classical emulation gives out precisely in the regime where the QPU is most interesting. That boundary is where neutral-atom hardware has to earn its keep.

EVERYDAY PICTURE

Describing a crowd. A state vector lists every possible arrangement of everyone: complete but astronomically long. An MPS records each person plus how tightly they hold hands with the next person. For a queue, that's short and accurate. For a mosh pit, there's no shortcut.

CHECK YOURSELF · PART 3 — tap a question to reveal the answer

Q Why does the adiabatic sweep start with negative detuning?

Negative δ makes excitations expensive, so the lowest-energy state is "all atoms in |g⟩", which is exactly the state the atoms start in.

Q Why is MIS on unit-disk graphs "native" to neutral atoms?

The blockade automatically forbids two connected atoms from both being excited, which is exactly the independence constraint. No extra encoding is needed.

Q Which QUBO matrices can be embedded directly, and why?

Those with positive off-diagonal entries, because the van der Waals interaction is always repulsive (positive). Even then, a 2D geometry may not reproduce every value exactly.

Q What does the bond dimension of an MPS measure?

How much entanglement the representation can hold across each cut of the chain. More entanglement needs a larger bond dimension and more compute.

PART 4 · THE PLATFORM MAP

How Pasqal's tools fit together

You now know the physics and the kinds of problems a neutral-atom QPU solves. Before writing code, it helps to see the whole platform at once: which pieces exist, what each one is for, and how a program travels from your laptop to a machine full of atoms.

YOU WILL LEARN

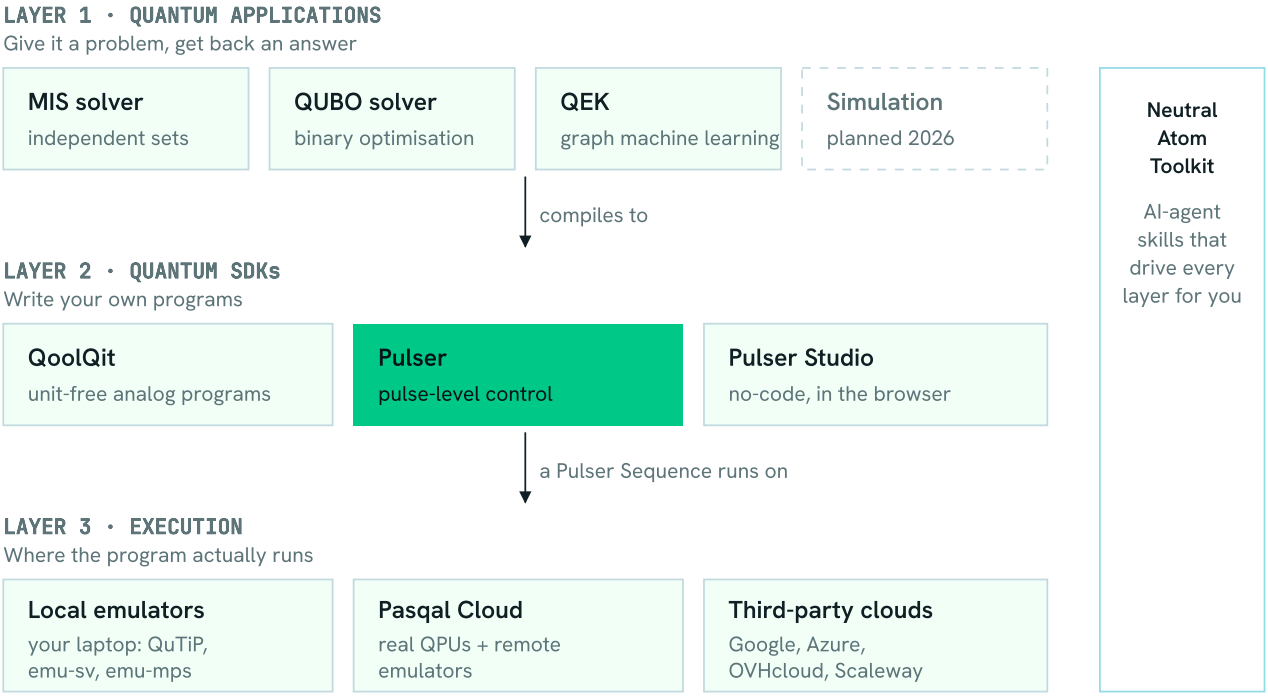
- The three layers of Pasqal's stack
- Which tool fits which person and problem
- How docs.pasqal.com is organised

TIME

- About 20 minutes
- No code yet

4.1 Three layers: applications, SDKs, execution

Pasqal's documentation hub groups everything into three layers, from "closest to your problem" down to "closest to the atoms". Higher layers are easier to use; lower layers give you more control. Every layer eventually produces the same thing: a **Pulser sequence**, the precise list of atom positions and laser pulses the hardware executes.



Pasqal's stack, as the docs hub organises it. Whatever you start from, the result is a Pulser sequence, which can run on a local emulator, on Pasqal Cloud, or through a partner cloud. Only the connection object changes; the sequence stays the same.

Quantum Applications	Quantum SDKs	Execution
Ready-made solvers. You hand over a graph or a matrix, and the library handles the embedding into atoms, the pulse design and the post-processing. No quantum knowledge needed to try them.	Libraries for writing programs yourself. **Pulser** is the foundation and talks in physical units; **QoolQit** sits on top and lets you think in unit-free terms; **Pulser Studio** is a visual editor.	Emulators that imitate the QPU on ordinary computers, the real QPUs, and the cloud services that give you access to both.

4.2 Which tool should you use?

Pick by what you are trying to do, not by how advanced the tool sounds. Most learners start with Pulser, because every other tool produces Pulser sequences in the end and understanding them makes everything else readable.

IF YOU WANT TO…	USE	INSTALL / OPEN
Understand exactly what the machine does, pulse by pulse	**Pulser**	`pip install pulser`
Design analog algorithms without juggling µm, ns and rad/µs	**QoolQit**	`pip install qoolqit`
Explore visually, teach, or prototype without code	**Pulser Studio**	pulserstudio.pasqal.cloud
Solve a maximum independent set (scheduling, placement, colouring)	**MIS solver**	`pip install maximum-independent-set`
Solve a binary optimisation problem written as a matrix	**QUBO solver**	`pip install qubo-solver`
Classify graphs or molecules with a quantum kernel	**QEK**	`pip install quantum-evolution-kernel`
Test on your own computer, free	**Local emulators**	included with Pulser; `pip install emu-sv emu-mps`
Run on a real QPU or a big cloud emulator	**Pasqal Cloud** or a partner cloud	`pasqal-cloud` (installed with Pulser)
Let an AI coding agent draft, emulate and submit experiments with you	**Neutral Atom Toolkit**	agent plugin (Part 7)

4.3 Finding your way around docs.pasqal.com

All of Pasqal's documentation lives on one site, docs.pasqal.com. Its sidebar mirrors the three layers above:

SIDEBAR SECTION	WHAT YOU FIND THERE
Quantum Applications	MIS, QEK and QUBO libraries; a Simulation library is listed as planned for 2026
Quantum SDKs	QoolQit, Pulser and Pulser Studio, each with tutorials and an API reference
Execution	QPU & Emulators (device specs, local emulators), Pasqal Cloud (accounts, jobs, billing), Third-party Cloud Providers
Neutral Atom Toolkit	The AI-agent skills for running experiments end to end

SIDEBAR SECTION	WHAT YOU FIND THERE
Onboarding	A guided three-step path: find the right tool, run your first experiment, track results on Pasqal Cloud
Product News	A changelog of new features and devices, for example the Canadian QPU going live (31 October 2025)

Two other sites matter. portal.pasqal.cloud is the web portal where you sign up, join projects and track jobs. community.pasqal.com hosts the free **Quantum Quest** course and links to the community Slack.

4.4 What changed recently

Pasqal's software moved quickly in 2025 and 2026. Many blog posts, videos and older notebooks still show the previous ways of doing things. If code you find online doesn't run, check it against this list first.

WATCH OUT · OLD VS CURRENT (OCTOBER 2026)

- **Cloud access:** the `pulser-pasqal` package is deprecated. Use `pasqal-cloud` and its `PasqalCloudConnection` instead of `PasqalCloud`, or the older `SDK(...)` object.
- **Emulator backends were renamed:** `EmuMPSBackend` became `RemoteMPSBackend`, and `EmuFreeBackendV2` became `RemoteEmuFreeBackend`. `EmuTNBackend` and `EmuFreeBackend` were removed.
- **Local simulation:** `QutipEmulator` is deprecated since Pulser 1.9. Use `QutipBackendV2` with an `EmulationConfig`.
- **Switching devices:** `switch_device` and `switch_register` became `with_new_device` and `with_new_register` (Pulser 1.6).
- **Qadence**, the digital-analog QML library featured in many 2023–24 tutorials, is marked "not actively maintained" and has left the docs hub. For new analog work, use **QoolQit**.
- **EMU-TN**, the tensor-network cloud emulator, is deprecated on Pasqal Cloud and Google Cloud. Use **EMU-MPS**.

This guide uses the current versions throughout: **Pulser 1.9.1** and **pasqal-cloud 0.24.0**, both released in September 2026 and requiring Python 3.10 or newer.

PART 5 · PULSER, HANDS ON

Programming atoms with Pulser

Pulser is Pasqal's open-source Python library for designing pulse sequences on neutral-atom devices. Everything else on the platform, from Pulser Studio to the solvers, ends up as a Pulser sequence. Once you can read one, you can read the whole platform.

YOU WILL LEARN

- The five objects of every Pulser program
- Units and conventions that trip everyone up
- How to write, draw, emulate and parametrize a sequence
- How to add realistic noise

YOU NEED

- Python 3.10 or newer
- About 3–4 hours, spread over a few sessions
- A Jupyter notebook helps (plots appear inline)

5.1 Install

Create a fresh virtual environment, then install the `pulser` package. It is a meta-package: it brings in `pulser-core` (registers, devices, pulses, sequences), `pulser-simulation` (a local emulator built on the QuTiP physics library) and `pasqal-cloud` (the connection to Pasqal Cloud).

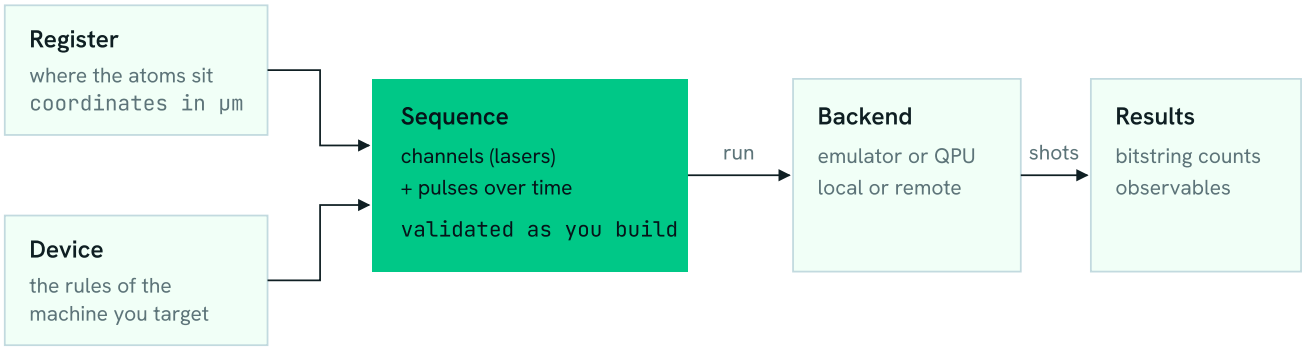
terminal

```
python -m venv pasqal-env
# Windows: pasqal-env\Scripts\activate    macOS/Linux: source pasqal-env/bin/activate
pip install pulser            # core + simulation + pasqal-cloud
pip install emu-sv emu-mps    # optional: faster, bigger local emulators
pip install jupyter matplotlib
```

Check it worked with `python -c "import pulser; print(pulser.__version__)"`. At the time of writing this prints `1.9.1`.

5.2 The mental model: five objects

Every Pulser program, however large, is built from the same five objects. Learn their roles once and the rest is detail.



The shape of every Pulser program. A sequence is always tied to one register and one device; the device checks every pulse you add, so mistakes fail early, at the line that caused them.

`Register`

A named set of atom positions in micrometres. The order you list atoms in fixes the order of bits in your results.

`Device`

A description of a machine: how many atoms, how close they may be, which lasers exist and how strong they can be. Pulser ships example devices; real ones are downloaded from the cloud.

`Sequence`

Your program. You declare **channels** (lasers available on the device) and add **pulses** to them, one after another.

`Pulse` & waveforms

A pulse is an amplitude shape, a detuning shape and a phase, all over the same duration. Waveforms describe the shapes.

Backend

Something that runs a sequence: `QutipBackendV2` on your laptop, `RemoteMPSBackend` in the cloud, `QPUBackend` on real atoms.

Results

Counts of measured bitstrings, plus any observables you asked an emulator to compute (occupations, energy, fidelity…).

5.3 Units and conventions

Pulser uses one fixed set of units, with no exceptions. Getting these wrong is the most common beginner bug, because the units for time and frequency look mismatched at first.

QUANTITY	UNIT IN PULSER	EXAMPLE
Duration	**nanoseconds** (whole numbers)	`1000` = 1 µs
Amplitude Ω (Rabi frequency)	**rad/µs**	`np.pi` ≈ 3.14 rad/µs
Detuning δ	**rad/µs**	`-10`
Phase φ	radians (constant per pulse)	`np.pi / 2`
Distances, blockade radius	**µm**	`5.0`
Interaction coefficient $C_6/\hbar$	rad·µm⁶/µs	`865723.02` (n = 60)
Noise rates	1/µs	`dephasing_rate=0.05`

QUANTITY	UNIT IN PULSER	EXAMPLE
Temperature	µK	`temperature=50`

KEY IDEA · THE MIXED UNITS WORK OUT

Durations are in nanoseconds, but frequencies are per *micro*second. A pulse of `1000` ns at Ω = π rad/µs has area π × 1 µs = π: exactly a π-pulse. Hardware datasheets often quote frequencies in MHz; multiply by 2π to get rad/µs. The example device's maximum amplitude, 12.566 rad/µs, is 2 MHz × 2π.

Which state is "1"?

Every atom starts in its ground state. In the standard `ground-rydberg` basis, an atom measured in the Rydberg state reads as `1`, and an atom in the ground state reads as `0`. Bits appear in the same order as the atoms in your register, so for atoms `q0, q1, q2` the bitstring `"010"` means only `q1` was excited.

BASIS	TWO LEVELS	DRIVEN BY	STARTS IN	READS AS 1
`ground-rydberg`	|g⟩, |r⟩	Rydberg channel (and DMM)	|g⟩	|r⟩
`digital`	|g⟩, |h⟩	Raman channel	|g⟩	|h⟩
`XY`	|0⟩, |1⟩ (two Rydberg states)	Microwave channel	|0⟩	|1⟩

Pasqal's cloud QPUs currently expose only the `ground-rydberg` basis through one global Rydberg channel. The other bases exist on example and virtual devices, so you can learn them in emulation.

5.4 Registers and layouts

A register is a dictionary of atom names and positions. Pulser offers shortcuts for common shapes.

```python
import numpy as np
import pulser
from pulser import Register

reg = Register({"q0": (0, 0), "q1": (5, 0), "q2": (2.5, 4.33)})   # by hand, in µm
sq  = Register.square(3, spacing=6.0, prefix="q")                 # 3x3 grid: q0 ... q8
tri = Register.triangular_lattice(rows=3, atoms_per_row=4, spacing=5.0, prefix="q")
hexr = Register.hexagon(layers=2, spacing=5.0, prefix="q")        # rings around a centre atom
pts = Register.from_coordinates([(0, 0), (7, 0), (14, 0)], prefix="q")

# Draw it, with lines between atoms that sit inside each other's blockade radius
R_b = pulser.AnalogDevice.rydberg_blockade_radius(2 * np.pi)      # µm, for Ω = 2π rad/µs
sq.draw(blockade_radius=R_b, draw_graph=True, draw_half_radius=True)
```

With `draw_half_radius=True`, each atom gets a circle of radius $R_b/2$; where two circles overlap, those atoms block each other. This picture is the fastest way to check that

your register encodes the graph you intended.

Layouts: why real hardware needs them

On the real machine, atoms can only sit where the optical system has made a trap. A **register layout** is the full set of trap positions. Your register then picks which traps to fill. Calibrating a new layout takes the Pasqal team time, so it is best to reuse **pre-calibrated layouts**.

```python
from pulser.register.special_layouts import TriangularLatticeLayout

device = pulser.AnalogDevice
print(device.requires_layout)            # True: a bare Register will not pass QPU checks
print(device.calibrated_register_layouts.keys())   # ['TriangularLatticeLayout(61, 5.0µm)']

# Option 1: let Pulser build a compliant layout around your register
reg_ok = sq.with_automatic_layout(device)

# Option 2: pick traps from a layout yourself
layout = TriangularLatticeLayout(n_traps=61, spacing=5)
reg_from_layout = layout.define_register(26, 35, 36, 27, qubit_ids=["a", "b", "c", "d"])

# Option 3: decide which traps to use later, at build time
mappable = layout.make_mappable_register(n_qubits=4)   # ids q0..q3
```

> WATCH OUT
>
> Pulser validates distances immediately. Putting atoms 2 µm apart on `AnalogDevice` raises `ValueError: The minimal distance between atoms in this device (5 µm) is not respected`. Read the error; it tells you which rule you broke.

5.5 Devices: the rules of the machine

A `Device` describes what a machine can and cannot do. Pulser includes example devices for learning. They enforce realistic constraints, but they are not exact datasheets: for real limits, fetch the live device from the cloud (Part 6).

SETTING	`AnalogDevice`	`DigitalAnalogDevice`	REAL ORION BETA QPU
Max atoms	80	100	100
Min distance between atoms	5 µm	4 µm	5 µm
Max distance from centre	38 µm	50 µm	35 µm
Rydberg level *n*	60	70	60
$C_6/\hbar$ (rad·µm⁶/µs)	865,723	5,420,159	865,723
Channels	rydberg_global	rydberg_global, rydberg_local, raman_local, dmm_0	rydberg_global
Max amplitude	12.57 rad/µs	15.71 rad/µs	from cloud specs
Max |detuning|	125.66 rad/µs	125.66 rad/µs	from cloud specs

SETTING	`AnalogDevice`	`DigitalAnalogDevice`	REAL ORION BETA QPU
Max sequence length	6,000 ns	none	6,000 ns
Requires a layout	yes	yes	yes

Two more devices are useful. `MockDevice` is unconstrained and has every channel type, including the microwave channel for XY mode. A `VirtualDevice` is a device you configure yourself; it lets you, for example, use 3D registers or reuse a channel twice. Print any device's limits with `device.print_specs()`.

5.6 Channels: the lasers you can use

A channel is one controllable laser (or microwave) source on the device. Its name tells you its type and how it addresses atoms: `rydberg_global` is a Rydberg laser that shines on all atoms equally; `raman_local` is a Raman laser aimed at chosen atoms.

CHANNEL TYPE	DRIVES	USED FOR
Rydberg (global or local)	$\|g\rangle \leftrightarrow \|r\rangle$	Analog computing; interactions via the blockade. The only channel on today's cloud QPUs.
Raman (global or local)	$\|g\rangle \leftrightarrow \|h\rangle$	Single-qubit gates in digital mode
Microwave (global)	$\|0\rangle \leftrightarrow \|1\rangle$ (two Rydberg states)	XY-model simulation with dipolar interactions
DMM (detuning map modulator)	extra detuning on chosen atoms	Giving atoms different "weights" without a local laser

Each channel has limits that every pulse must respect. On `AnalogDevice`'s Rydberg channel: durations must be a multiple of the **clock period** (4 ns) and at least 16 ns long, amplitude at most 12.57 rad/µs, and detuning within ±125.66 rad/µs.

Modulation bandwidth

Real lasers can't change instantly. Pulser models this with `mod_bandwidth` (8 MHz here): your programmed "input" is smoothed into an "output". `seq.draw()` shows both. Very short square pulses may never reach their programmed height.

EOM mode

A special mode that gives sharper square pulses (40 MHz bandwidth on `AnalogDevice`), entered with `enable_eom_mode`. Useful when you need crisp on/off control.

DMM and detuning maps

Assign each atom a weight between 0 and 1, then apply a negative detuning scaled by those weights. This is how weighted problems (MWIS) put different "rewards" on different atoms.

SLM mask

Detunes chosen atoms far off resonance so a global pulse skips them. The standard trick for preparing an uneven starting state.

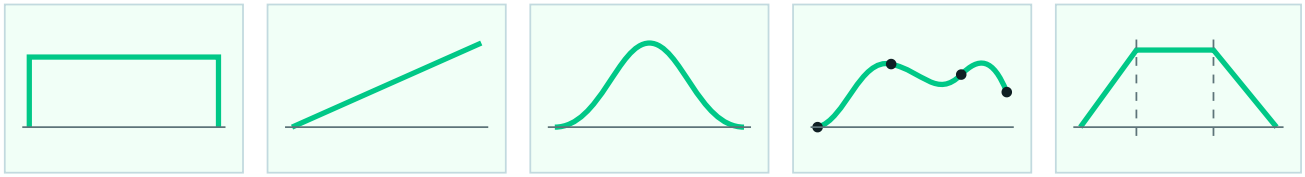
5.7 Pulses and waveforms

A `Pulse` bundles three things over one duration: an **amplitude waveform** Ω(t), a **detuning waveform** δ(t), and a fixed **phase** φ. Both waveforms must have the same duration.

```python
from pulser import Pulse
from pulser.waveforms import (ConstantWaveform, RampWaveform, BlackmanWaveform,
                              InterpolatedWaveform, CompositeWaveform)

# Pulse(amplitude, detuning, phase, post_phase_shift=0.0)
p1 = Pulse(BlackmanWaveform(500, np.pi), RampWaveform(500, -10, 10), phase=0.0)

# Shortcuts when one of the two is constant
p2 = Pulse.ConstantPulse(1000, np.pi, 0.0, 0.0)                    # duration, Ω, δ, φ
p3 = Pulse.ConstantDetuning(RampWaveform(252, 0, 12), -18.8, 0.0)   # shaped Ω, fixed δ
p4 = Pulse.ConstantAmplitude(12, RampWaveform(800, -18.8, 12.5), 0.0)  # fixed Ω, shaped δ
p1.draw()
```



Constant hold a value

Ramp straight line

Blackman smooth bell, set by area

Interpolated smooth curve through points

Composite pieces joined end to end

The waveform toolbox. All take a duration in nanoseconds first. Kaiser (another smooth, area-defined bell) and Custom (one value per nanosecond) complete the set.

WAVEFORM	SIGNATURE	WHEN TO USE IT
`ConstantWaveform`	`(duration, value)`	Holding a level
`RampWaveform`	`(duration, start, stop)`	Sweeps, rises and falls
`BlackmanWaveform`	`(duration, area)` or `.from_max_val(max_val, area)`	Smooth pulses with an exact area, such as clean π-pulses
`KaiserWaveform`	`(duration, area, beta=14.0)`	Like Blackman, with a tunable shape
`InterpolatedWaveform`	`(duration, values, times=None)`	Smooth custom shapes; handy for optimising pulse shapes
`CompositeWaveform`	`(*waveforms)`	Joining several shapes into one
`CustomWaveform`	`(samples)`	Full control: one value per nanosecond

KEY IDEA · PULSE AREA

The **area** of an amplitude waveform is ∫Ω dt, in radians. For one atom on resonance, area π flips |g⟩ to |r⟩ (a **π-pulse**), area π/2 makes an equal superposition, and area 2π brings it back. `BlackmanWaveform(1000, np.pi)` has area exactly π, with a peak of about 7.49 rad/μs. Blackman shapes are preferred to square pulses because their smooth edges avoid exciting unwanted frequencies.

5.8 Building a sequence

A sequence ties a register to a device. You declare the channels you will use, then add pulses to them in order.

```python
from pulser import Sequence
from pulser.devices import MockDevice

seq = Sequence(reg, MockDevice)                # MockDevice: unconstrained, good for learning
print(seq.available_channels.keys())           # what this device offers

seq.declare_channel("ryd", "rydberg_global")   # your name, the device's channel id
seq.add(p2, "ryd")                             # pulses run one after another
seq.delay(200, "ryd")                          # wait 200 ns
seq.add(p1, "ryd")

seq.measure("ground-rydberg")                  # optional: measurement basis
print(seq.get_duration(), "ns")
print(seq)                                     # text schedule
seq.draw()                                     # plot: amplitude, detuning, phase
```

When a sequence uses several channels (for example a global and a local laser), `add` takes a `protocol` argument that decides how a new pulse lines up with the others:

`"min-delay"` (default)	`"wait-for-all"`	`"no-delay"`
Wait only as long as needed to avoid clashing with other channels acting on the same atoms.	Wait until every channel is free, then start.	Start immediately, even if that overlaps other pulses.

Other useful methods: `align(*channels)` lines channels up in time; `target(qubits, channel)` moves a local channel's aim; `phase_shift(phi, *qubits)` applies a "virtual Z" rotation for free; `with_new_device(device)` re-checks the whole sequence against another device; `to_abstract_repr()` saves it as JSON, which is how sequences travel to the cloud.

5.9 Your first program: flip one atom

Start with the simplest possible experiment: one atom, one π-pulse, and a check that it ends up in the Rydberg state every time.

python · tested on Pulser 1.9.1

```python
import numpy as np
import pulser

reg = pulser.Register.from_coordinates([(0, 0)], prefix="q")
seq = pulser.Sequence(reg, pulser.AnalogDevice)
seq.declare_channel("rydberg_global", "rydberg_global")

# Area = π: 1000 ns at Ω = π rad/µs, no detuning, phase 0
seq.add(pulser.Pulse.ConstantPulse(1000, np.pi, 0.0, 0.0), "rydberg_global")

results = pulser.backends.QutipBackendV2(seq).run()
print(results.final_bitstrings)
```

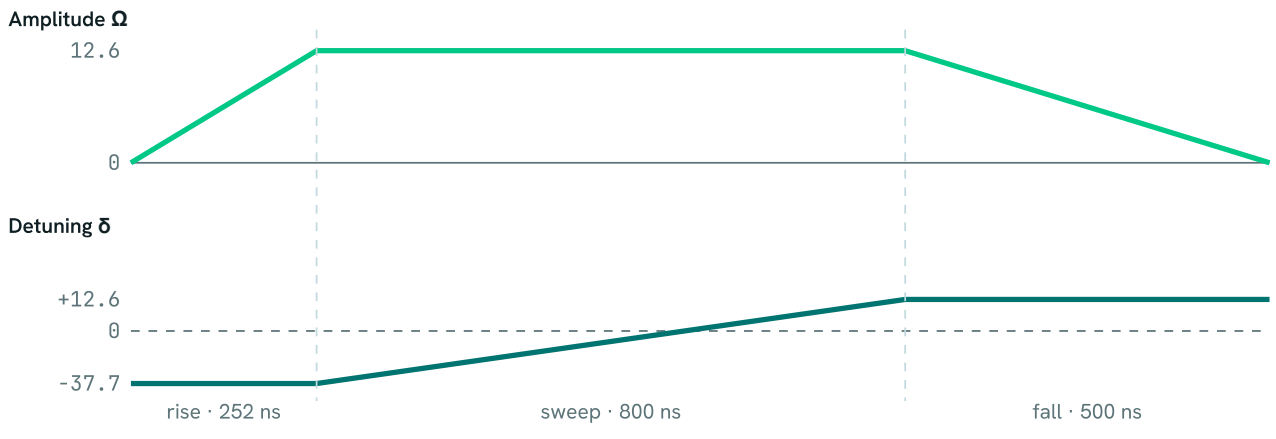
output

```
Counter({'1': 1000})
```

All 1000 simulated shots read `1`: a perfect π-pulse. Now experiment. Change the duration to 500 ns and you should see roughly half `0` and half `1`. Change it to 2000 ns and you should be back to all `0`. You are tracing a Rabi oscillation, the same curve as in the lab in Part 2.

5.10 A real analog program: an antiferromagnet on a 3×3 grid

This is Pulser's classic tutorial and a template for many analog algorithms. Nine atoms sit on a square grid. We slowly sweep the laser detuning from negative to positive, and the atoms settle into a **checkerboard** pattern: excited, not excited, excited… This is called an **antiferromagnetic (AFM)** state, and it is the lowest-energy arrangement when neighbours block each other.



The AFM schedule (values in rad/µs, total 1,552 ns). Turn the laser on while far below resonance, sweep the detuning slowly through zero to a positive value, then turn the laser off. Slow sweeps let the atoms follow their lowest-energy state, which is the adiabatic idea from Part 3.

python · tested on Pulser 1.9.1

```python
import numpy as np
import pulser

U = 2 * np.pi                    # target neighbour interaction, rad/µs
Omega_max = 2.0 * U              # 12.57 rad/µs: the device maximum
delta_0, delta_f = -6 * U, 2 * U
t_rise, t_fall = 252, 500
t_sweep = int((delta_f - delta_0) / (2 * np.pi * 10) * 1000)   # 800 ns

device = pulser.AnalogDevice
R = (device.interaction_coeff / U) ** (1 / 6)    # spacing so that C6/R^6 = U  → 7.19 µm
reg = pulser.Register.square(3, R, prefix="q")

seq = pulser.Sequence(reg, device)
seq.declare_channel("rydberg_global", "rydberg_global")
seq.add(pulser.Pulse.ConstantDetuning(
    pulser.RampWaveform(t_rise, 0.0, Omega_max), delta_0, 0.0), "rydberg_global")
seq.add(pulser.Pulse.ConstantAmplitude(
    Omega_max, pulser.RampWaveform(t_sweep, delta_0, delta_f), 0.0), "rydberg_global")
seq.add(pulser.Pulse.ConstantDetuning(
    pulser.RampWaveform(t_fall, Omega_max, 0.0), delta_f, 0.0), "rydberg_global")

print(seq.get_duration(), "ns")          # 1552 ns, well inside the 6000 ns limit

results = pulser.backends.QutipBackendV2(seq).run()
counts = results.final_bitstrings
print(sorted(counts.items(), key=lambda kv: -kv[1])[:3])
```

output

```
1552 ns
[('101010101', 105), ('101000101', 45), ('111000101', 22)]
```

The most common outcome, `101010101`, is the checkerboard: atoms 0, 2, 4, 6 and 8 (the corners and the centre) excited, the edge atoms not. Other outcomes appear because the sweep is finite and the edges of a small grid are imperfect. That is normal: on a QPU you always study a **distribution**, never a single answer.

KEY IDEA · THE RECIPE BEHIND MOST ANALOG ALGORITHMS

Choose atom positions so that the interactions encode your problem. Start the laser far below resonance, where "all atoms in |g⟩" is the lowest-energy state. Sweep slowly to a point where the lowest-energy state is the answer you want. Measure many times. The MIS and QUBO solvers in Part 7 automate exactly these steps.

5.11 Parametrized sequences

When you want to try many variants of one program (different amplitudes, durations or atom choices), declare **variables** instead of hard-coding numbers. You build the concrete sequence later. This is also how one cloud batch can hold many related jobs.

```python
seq = pulser.Sequence(reg, pulser.AnalogDevice)
seq.declare_channel("ryd", "rydberg_global")

omega = seq.declare_variable("omega")                # a float
t = seq.declare_variable("t", dtype=int)             # an integer duration in ns
seq.add(pulser.Pulse.ConstantPulse(t, omega, 0.0, 0.0), "ryd")

print(seq.is_parametrized())                         # True: can't be run or drawn yet
built = seq.build(omega=np.pi, t=1000)               # now it is a normal sequence
built.draw()
```

Variables support arithmetic (`delta_0 = -6 * omega`), so whole schedules can hang off one or two numbers. A `MappableRegister` works the same way for atom positions: you choose the traps at `build(qubits={...})` time.

5.12 Emulating and measuring observables

An emulator can do something a real QPU cannot: show you the quantum state *during* the program. You ask for **observables** and say at which moments to compute them. Times are fractions of the total duration: `0.0` is the start, `0.5` the midpoint, `1.0` the end.

```python · tested on Pulser 1.9.1
import pulser_simulation
from pulser.backend import BitStrings, Occupation, Energy

config = pulser_simulation.QutipConfig(
    observables=[BitStrings(num_shots=500), Occupation(), Energy()],
    default_evaluation_times=[0.0, 0.5, 1.0],
)
res = pulser_simulation.QutipBackendV2(seq, config=config).run()   # seq = the AFM sequence from 5.10

print(res.get_result_tags())                 # ['bitstrings', 'occupation', 'energy']
print(res.get_result("occupation", 1.0))     # probability each atom is excited, at the end
print(res.get_result("energy", 1.0))
```

```output (AFM sequence from 5.10)
['bitstrings', 'occupation', 'energy']
[0.792 0.393 0.792 0.393 0.327 0.393 0.792 0.393 0.792]
15.505...
```

The occupations show the checkerboard tendency directly: corners about 0.79, edges about 0.39, and the centre about 0.33. The centre is lower than the corners because it has four neighbours competing with it.

OBSERVABLE	RETURNS
`BitStrings(num_shots=…)`	Sampled measurement outcomes, like a QPU would give
`Occupation()`	Probability that each atom is in the "1" state

OBSERVABLE	RETURNS
`CorrelationMatrix()`	How often pairs of atoms are excited together
`Energy()`, `EnergyVariance()`	The energy of the state under the Hamiltonian
`Fidelity(state)`	Overlap with a target state you supply
`Expectation(operator)`	The average value of any operator you define
`StateResult()`	The full quantum state (local emulators only)

Every backend that runs on your machine or the cloud takes the same shape: `Backend(sequence, config=...)` and then `.run()`. Swap `QutipBackendV2` for `emu_sv.SVBackend` or `emu_mps.MPSBackend` when you need more atoms; see Part 6.

> WATCH OUT · OLD TUTORIALS
>
> Notebooks written before 2026 usually call `QutipEmulator.from_sequence(seq).run()` and then `sample_final_state()`. That still works in Pulser 1.9 but prints a deprecation warning. Prefer `QutipBackendV2`.

5.13 Adding realistic noise

A perfect emulator is optimistic. Real atoms suffer the noise sources from Part 2. A `NoiseModel` lets you switch each one on, so you can see how your algorithm will fare on hardware before spending QPU time.

```python
from pulser import NoiseModel
from pulser.backend import EmulationConfig

noise = NoiseModel(
    state_prep_error=0.005,   # atom missing at the start
    p_false_pos=0.02,         # ground atom read as 1
    p_false_neg=0.01,         # Rydberg atom read as 0
    temperature=50.0,         # μK: Doppler shifts from atomic motion
    laser_waist=175.0,        # μm: beam is weaker at the edges
    amp_sigma=0.05,           # 5% shot-to-shot amplitude jitter
)
print(noise.noise_types)      # ('SPAM', 'amplitude', 'doppler')

config = EmulationConfig(noise_model=noise, n_trajectories=50,
                         observables=[BitStrings(num_shots=1000)])
noisy = pulser_simulation.QutipBackendV2(seq, config=config).run()
```

NOISE FAMILY	PARAMETERS	WHAT IT MODELS
SPAM	`state_prep_error`, `p_false_pos`, `p_false_neg`	Preparation and measurement mistakes
Doppler	`temperature`	Moving atoms see a shifted laser frequency
Amplitude	`laser_waist`, `amp_sigma`	Uneven or fluctuating laser power

NOISE FAMILY	PARAMETERS	WHAT IT MODELS
Detuning	`detuning_sigma`, `detuning_hf_psd`	Fluctuating laser frequency
Dephasing / relaxation	`dephasing_rate`, `relaxation_rate`	Loss of quantum coherence; decay from |r⟩
Depolarizing, effective	`depolarizing_rate`, `eff_noise_rates`	Generic error channels you define
Register	`temperature`, `trap_waist`, `trap_depth`	Atoms jiggling around their trap centres

Real cloud devices ship their own noise model. Setting `prefer_device_noise_model=True` in the config uses it, which is the most honest preview of QPU results.

5.14 Pulser Studio: the same ideas, no code

Pulser Studio is a browser-based, no-code editor for neutral-atom programs. You place atoms on a canvas, draw pulses channel by channel, and run small registers on an emulator inside your browser. Larger runs go to Pasqal Cloud's GPU emulators. A code panel generates the equivalent Pulser Python as you work, so it is also a good way to learn the API: build something visually, then read the code it produces.

Its docs include two starter tutorials (Rabi oscillations, and the antiferromagnetic state from 5.10) and a tour of the interface. Full features need a Pasqal Cloud account.

5.15 Official Pulser tutorials, in a sensible order

The tutorials live in the Pulser docs and in the `tutorials/` folder of the GitHub repository. After this guide, work through them in this order:

1. **Programming with Pulser** (getting started): the π-pulse and the AFM state you just met.
2. **Execution on an emulator**: backends, `EmulationConfig`, observables.
3. **Register layouts** and **Virtual devices**: what real hardware requires.
4. **Parametrized sequences**, **Composite** and **Interpolated waveforms**: building flexible programs.
5. **Output modulation and EOM mode**: how real lasers smooth your pulses.
6. **QAA to solve a QUBO problem** and **QAA to solve a MWIS problem**: full applications.
7. **Bayesian optimisation for AFM state preparation**: tuning a schedule in a loop.

3 **Spin chain of 3 atoms in XY mode**

, **DMM**, **SLM mask**, **Phase shifts**: the more advanced controls.

9 **Execution on a QPU**

: once you have cloud access.

Older notebooks such as *intro to the Rydberg blockade*, the *control-Z gate* and the noise simulations have moved to a `tutorials/retired/` folder. They are still worth reading for the physics, but their code may use older APIs.

CHECK YOURSELF · PART 5 — tap a question to reveal the answer

Q A pulse has duration 500 and amplitude π. What fraction of the way from |g⟩ to |r⟩ does it take one atom?

Area = π rad/μs × 0.5 μs = π/2. That is half a π-pulse, so the atom ends in an equal superposition: about 50% of shots read 1.

Q Your register has atoms a, b, c (in that order). What does the bitstring "001" mean?

Only atom c was measured in the Rydberg state; a and b were in the ground state.

Q Why does a hand-made Register fail when you target a real QPU?

Real devices require a register layout (`requires_layout=True`): atoms must sit in calibrated trap positions. Use `with_automatic_layout(device)` or build the register from a pre-calibrated layout.

Q In the AFM program, why is the spacing set to (C₆/U)^(1/6)?

Because the interaction between two atoms is C_6/R^6. Choosing R that way makes the neighbour interaction exactly U, the energy scale the detuning sweep was designed around.

Q What does evaluation time 0.5 mean for an observable?

The halfway point of the whole sequence. Evaluation times are fractions of the total duration, not nanoseconds.

PART 6 · EMULATORS AND PASQAL CLOUD

From your laptop to real atoms

A Pulser sequence can run in three kinds of places: an emulator on your own computer, a bigger emulator in the cloud, or a real QPU. This part explains each option, how Pasqal Cloud organises your work into projects, batches and jobs, and what it costs.

YOU WILL LEARN

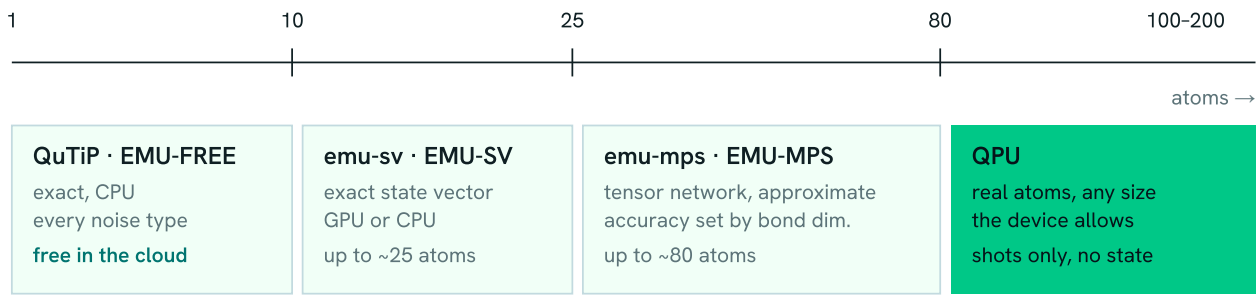
- Which emulator to use for how many atoms
- Pasqal's QPUs and their limits
- Projects, batches, jobs and shots
- The current `pasqal-cloud` SDK, and credits

YOU NEED

- Part 5 under your belt
- A free Pasqal Cloud account for the cloud sections (portal.pasqal.cloud)

6.1 Choosing an emulator

An emulator is ordinary software that calculates what the atoms would do. It is free or cheap, fast for small problems, and it can show you things a QPU never can, such as the full quantum state midway through. Its limit is size: the memory needed doubles with every atom you add (Part 3 explains why). Pasqal offers three emulation methods, each available locally and in the cloud.



Pick the emulator by problem size. The limits are the ones Pasqal states for its cloud emulators; local runs depend on your hardware. Approximate means the answer gets less reliable as entanglement grows; Part 3 explains the trade-off.

METHOD	LOCAL BACKEND (YOUR COMPUTER)	CLOUD BACKEND	NOTES
QuTiP (exact)	`QutipBackendV2` in `pulser-simulation`	`RemoteEmuFreeBackend` → EMU-FREE	Most complete noise support. EMU-FREE is free, CPU-only, about 10 qubits.
State vector (exact)	`SVBackend` in `emu-sv`	`RemoteSVBackend` → EMU-SV	About 25 qubits, GPU-accelerated, any 1D/2D/3D positions.
Matrix product state (approximate)	`MPSBackend` in `emu-mps`	`RemoteMPSBackend` → EMU-MPS	Up to 80 qubits in the cloud; works best for 1D chains (~100) and modest 2D (~40). No local addressing.

METHOD	LOCAL BACKEND (YOUR COMPUTER)	CLOUD BACKEND	NOTES
Tensor network (legacy)	none	EMU-TN	Deprecated on Pasqal Cloud and Google Cloud; still offered via OVHcloud and Scaleway.

KEY IDEA · REHEARSE WITH `mimic_qpu=True`

Emulators are more forgiving than hardware by default. Pass `mimic_qpu=True` to any emulator backend and it applies the same checks a QPU job would face: layouts, atom count, channel limits, maximum runs. It is the cheapest way to make sure your first real submission won't be rejected.

6.2 Running the tensor-network emulator yourself

`emu-mps` stores the quantum state as a **matrix product state**: a chain of small tensors instead of one enormous list of numbers. The size of those tensors, the **bond dimension**, sets both the accuracy and the cost. Low entanglement means small bonds and fast runs; high entanglement needs big bonds.

python

```python
# pip install emu-mps   (this also updates pulser-core to a compatible version)
from emu_mps import MPSBackend, MPSConfig
from pulser.backend import BitStrings, Occupation

config = MPSConfig(
    dt=4,                # time step in ns; keep evaluation times on multiples of dt
    max_bond_dim=256,    # cap on tensor size: higher = more accurate, slower
    precision=1e-5,      # truncation accuracy
    observables=[BitStrings(num_shots=1000), Occupation()],
)
results = MPSBackend(seq, config=config).run()
print(results.final_bitstrings.most_common(3))
```

MPSCONFIG OPTION	DEFAULT	MEANING
`dt`	10.0	Time step (ns) over which pulses are treated as constant. Observables are only computed at times that are multiples of `dt`.
`precision`	1e-5	How much accuracy you accept losing at each truncation step
`max_bond_dim`	1024	Hard cap on the bond dimension
`num_gpus_to_use`	None	GPUs to spread the tensors over; `0` forces CPU
`optimize_qubit_ordering`	True	Reorders atoms along the chain for better accuracy
`interaction_cutoff`	0.0	Ignore interactions weaker than this to speed things up
`solver`	TDVP	TDVP for real-time evolution; DMRG is typically used for adiabatic sequences
`autosave_dt`	inf	Save progress every so many seconds, so long runs can resume

WATCH OUT

In the emu-mps docs' own words: if the bond dimension reaches the maximum you set, the accuracy of the rest of the time evolution can no longer be guaranteed. Check the run's statistics and logs, and treat results near the cap with suspicion.

6.3 The QPUs

Pasqal's machines belong to the **Orion** family. The first generation, Orion Alpha, carried the name **Fresnel**, and cloud device names still start with it. The documentation lists six devices:

DEVICE NAME	GENERATION	MAX QUBITS	LOCATION	WHO CAN USE IT
`FRESNEL`	Orion Beta	100	France	Pasqal Cloud, Google Cloud, Azure Quantum, OVHcloud, Scaleway
`FRESNEL_CAN1`	Orion Beta	100	Canada	Pasqal Cloud, Google Cloud, Azure Quantum, OVHcloud, Scaleway
`FRESNEL_SA1`	Orion Gamma	140–200	Saudi Arabia	Pasqal Cloud, Saudi region only
`RUBY`	Orion Beta	100	France (CEA/TGCC)	HPC centre users only
`JADE`	Orion Beta	100	Germany (Jülich)	HPC centre users only
`SOL`	Orion Gamma	140	Italy (CINECA)	HPC centre users only

The published specifications for the two generations:

PARAMETER	ORION BETA	ORION GAMMA
Max atoms	100	200
Min distance between atoms	5 µm	5 µm
Max distance from centre	35 µm	70 µm
Rydberg level	60	60
Max sequence duration	6,000 ns	6,000 ns
Max shots per job	500	500
Max traps in a layout / max filling	200 / 45%	200 / 45%
Channels	Rydberg global	Rydberg global
Dimensions	1D and 2D	1D and 2D

Two things stand out. First, the only channel is a **global Rydberg laser**, so today's public QPUs are **analog** machines: every atom feels the same pulse, and you program them through atom positions and pulse shapes. Second, a layout may be at most 45% filled,

which leaves spare traps so that randomly loaded atoms can be rearranged into your register.

```python
from pasqal_cloud import PasqalCloudConnection

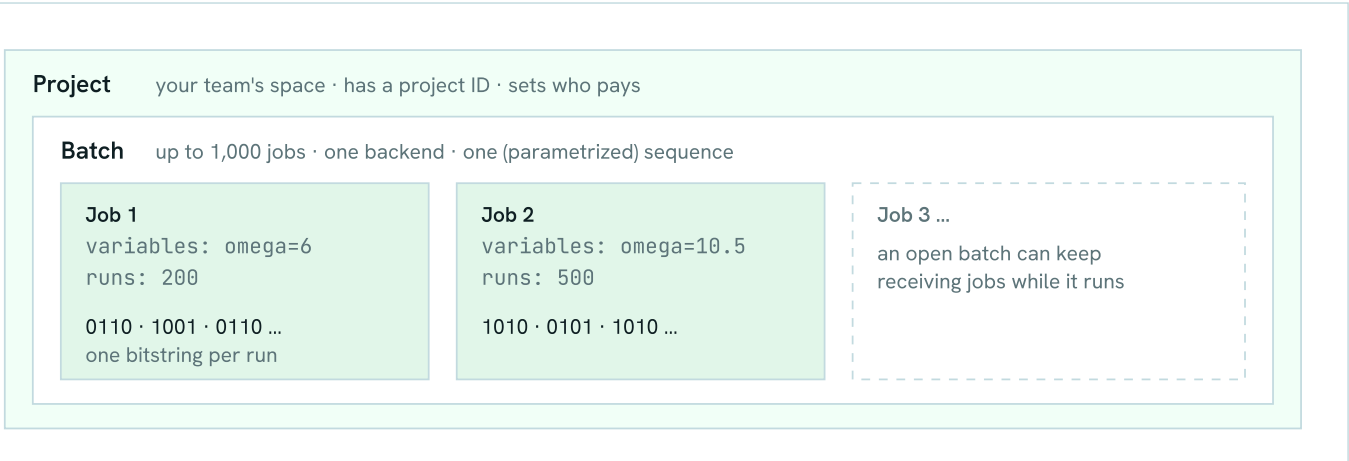
connection = PasqalCloudConnection(username="you@example.com", project_id="your-project-id")
devices = connection.fetch_available_devices()     # the live specs: always trust these
print(devices.keys())                              # e.g. dict_keys(['FRESNEL', 'FRESNEL_CAN1'])
fresnel = devices["FRESNEL_CAN1"]
fresnel.print_specs()
print(fresnel.max_runs, fresnel.max_atom_num, fresnel.pre_calibrated_layouts)
```

WATCH OUT

Numbers in documentation pages, tutorials and this guide can drift as machines are upgraded. The Pasqal docs say the portal's Devices page and the specs returned by the SDK take precedence. Build your sequence against the fetched device, not a hard-coded one.

6.4 Pasqal Cloud: projects, batches, jobs and shots

Pasqal Cloud is the hosted service that runs your sequences on QPUs and cloud emulators. You use it through a web **portal** (portal.pasqal.cloud, or portal.sa.pasqal.cloud for the Saudi region) and through the `pasqal-cloud` Python package. Work is organised in four nested levels:



How cloud work is organised. A **run** (also called a shot or repetition) is one execution that returns one bitstring. A **job** is a number of runs with fixed variable values. A **batch** groups jobs on the same backend. A **project** is the collaboration space you must belong to before you can submit anything.

Account and project

Sign up on the portal. To submit jobs you must be a member of at least one project; a project administrator invites members and chooses their role. Your **project ID** (under *Projects* in the portal) is required in every SDK connection and decides which credits are used.

Regions

There is a global region and a Saudi Arabia region with its own portal. In code, pass `region="sa"` to reach it; the default is the global region.

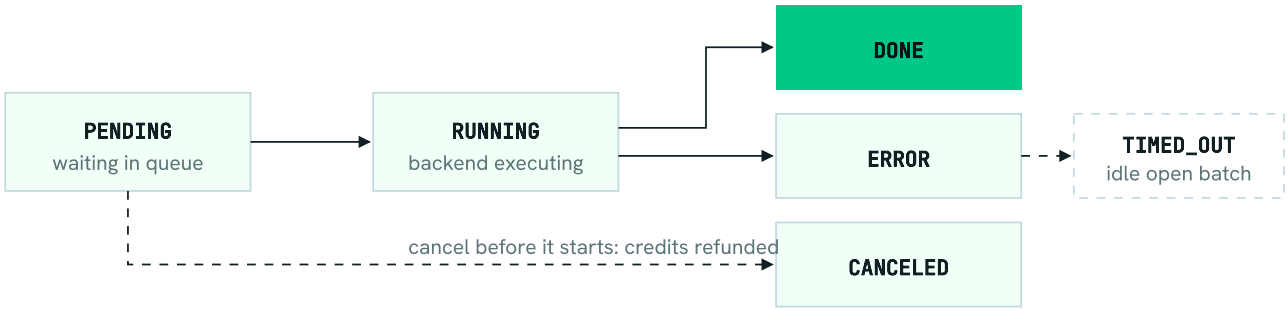
Batch rules

At most 1,000 jobs per batch, run in the order submitted. Jobs in one batch should share one sequence; jobs with different registers must at least share one trap layout (use a mappable register).

What the portal shows

Your jobs and their status, logs, retries, results to download as JSON, the live device list, your projects and your profile.

6.5 The life of a batch



Statuses you will see. A batch is `DONE` once every job in it has finished, whether a job succeeded, failed with `ERROR`, or was `CANCELED`. Canceled jobs are refunded.

6.6 The `pasqal-cloud` SDK

The current SDK (version 0.24.0, installed with Pulser) has a simple shape: create one **connection**, wrap your sequence in a **backend**, call `run`. The same sequence runs on an emulator or a QPU; only the backend class changes.

python · first cloud job (free EMU-FREE emulator)

```python
from pulser import Pulse, Register, Sequence
from pulser.devices import DigitalAnalogDevice
from pasqal_cloud import PasqalCloudConnection, RemoteEmuFreeBackend

# Leave out the password: the SDK will ask for it when you run the script.
connection = PasqalCloudConnection(username="you@example.com",
                                   project_id="your-project-id")   # add region="sa" for KSA

register = Register.square(2, spacing=5, prefix="q")
sequence = Sequence(register, DigitalAnalogDevice)
sequence.declare_channel("rydberg", "rydberg_global")
omega_max = sequence.declare_variable("omega_max")
sequence.add(Pulse.ConstantPulse(100, omega_max, 2, 0.0), "rydberg")

# Two jobs from one parametrized sequence
jobs = [{"runs": 20, "variables": {"omega_max": 6}},
        {"runs": 50, "variables": {"omega_max": 10.5}}]

backend = RemoteEmuFreeBackend(sequence=sequence, connection=connection)
results = backend.run(job_params=jobs, wait=True)

for job_id, result in results.get_available_results().items():
    print(job_id, result.final_bitstrings)      # e.g. {'0001': 4, '0101': 7, ...}
```

To use real hardware, build the sequence on a fetched device with a calibrated layout, then swap the backend class:

python · the same idea on a QPU

```python
from pulser.backends import QPUBackend, RemoteMPSBackend

device = connection.fetch_available_devices()["FRESNEL_CAN1"]
register = Register.square(5, 5).with_automatic_layout(device)   # or a pre-calibrated layout
sequence = Sequence(register, device)
sequence.declare_channel("rydberg", "rydberg_global")
omega_max = sequence.declare_variable("omega_max")
sequence.add(Pulse.ConstantPulse(100, omega_max, 2, 0.0), "rydberg")

jobs = [{"runs": 100, "variables": {"omega_max": 6}}]

# 1. Dry run: an emulator that applies the QPU's own checks
RemoteMPSBackend(sequence=sequence, connection=connection, mimic_qpu=True).run(job_params=jobs, wait=True)

# 2. The real thing
results = QPUBackend(sequence=sequence, connection=connection).run(job_params=jobs, wait=True)
print(results.batch_id)
print(results.results)
```

Signing in

Username and password (keep the password in an environment variable, never in code), an interactive prompt, a custom token provider, or service-account credentials for automation. Behind the scenes, Pasqal Cloud uses Auth0 tokens that last 24 hours.

Picking up results later

Note the `batch_id`. Pulser's `RemoteResults(batch_id, connection)` reconnects to it in a new session, and the portal lets you download results as JSON.

Hardware checks

Before running, a QPU checks that your channels exist on the device, that the register comes from a layout, and that spacing and atom count are within limits.

6.7 Open batches for feedback loops

Variational algorithms (Part 3) need to look at results before choosing the next parameters. An **open batch** keeps the backend reserved for you, so each new job runs straight away instead of rejoining the queue.

python

```python
backend = QPUBackend(sequence=sequence, connection=connection)

with backend.open_batch():
    first = backend.run(job_params=[{"runs": 50, "variables": {"omega_max": 9.5}}], wait=True)
    # ... look at `first`, decide what to try next ...
    backend.run(job_params=[{"runs": 50, "variables": {"omega_max": 10.0}},
                            {"runs": 50, "variables": {"omega_max": 10.5}}], wait=True)
# the batch closes automatically here
```

An open batch that receives no new job for a few minutes is stopped with status `TIMED_OUT`, so keep your classical step quick.

Managing work: the lower-level client

For housekeeping, `connection.cloud_client` gives you a `PasqalCloudClient`. It can list batches and jobs with filters (`get_batches`, `get_jobs`), tag batches (`set_batch_tags`, which replaces existing tags), cancel work (`cancel_batches`, `cancel_jobs`), rerun failed jobs in a closed batch (`rebatch`), switch project (`switch_to_project`) and download device specs (`get_device_specs_dict`).

6.8 Credits, pricing and the free tier

The free Explorer offer

Develop with Pulser, submit and track jobs, download results, and use Pulser Studio and the community platform, all free. The limits: only the **EMU-FREE** emulator, CPU only, at most 10 jobs at once, and a low-priority queue. No QPU access.

Getting QPU time

Contact Pasqal for a contract and credits, or pay as you go through the Google Cloud Marketplace. Partner clouds (next section) bill through their own accounts.

Credit pools

Direct contracts give you pools of credits, valid for one year by default. Each contract sets one price for emulators and one for QPUs, charged per hour or per shot.

How a submission is checked

A job is accepted only if its estimated cost is below your remaining credits: valid credits minus what completed jobs used minus estimates for pending jobs. Under hourly QPU pricing, each shot is estimated at 4 seconds, a deliberately cautious figure.

When a job finishes, its real cost is deducted from the pools that expire first, and the reservation is released. EMU-FREE costs nothing. Canceled jobs are refunded. Pasqal does not publish its rates; you see them in your contract and on the portal's offers page.

6.9 Partner clouds

You can also reach Pasqal's machines through four other cloud providers. The same Pulser sequence works everywhere; only the connection object changes. Usage on partner clouds is billed pay-as-you-go by that provider, with no Pasqal credit pools involved. (AWS Braket is not one of the routes.)

BACKEND	GOOGLE CLOUD	AZURE QUANTUM	OVHCLOUD	SCALEWAY
Access	Portal and SDK	SDK, via an Azure Quantum workspace	SDK (`OVHConnection`)	SDK
`FRESNEL`, `FRESNEL_CAN1`	yes	yes	yes	yes
Saudi QPU	no	no	no	no
EMU-FREE, EMU-MPS	yes	yes	yes	yes
EMU-SV	yes	yes	no	yes
EMU-TN (legacy)	no	no	yes	yes

Each provider has its own "create a connection" guide under Third-party Cloud Providers in the docs. Supercomputing centres (Jülich, CEA/TGCC, CINECA) give their

users direct access to on-site QPUs through ordinary job schedulers such as Slurm; remote emulators are not offered there, so users emulate locally.

6.10 Old code, new names

If you inherit code from an older tutorial, this table tells you what to change.

OLD (`pulser-pasqal` ERA)	NOW (`pasqal-cloud` 0.24)
`from pulser_pasqal import PasqalCloud`	`from pasqal_cloud import PasqalCloudConnection`
`SDK(username=..., password=..., project_id=...)`	`PasqalCloudConnection(...)`; the old style survives only on the lower-level `PasqalCloudClient`
`EmuMPSBackend`	`RemoteMPSBackend`
`EmuSVBackend`	`RemoteSVBackend`
`EmuFreeBackendV2`	`RemoteEmuFreeBackend`
`EmuFreeBackend`, `EmuTNBackend`	removed
`EmulatorType.EMU_TN` in `create_batch`	`DeviceTypeName` on `PasqalCloudClient.create_batch`, or simply a backend class

6.11 Checklist before your first QPU job

1. **Fetch the live device**
   with `connection.fetch_available_devices()` and build the sequence on it.
2. **Use a calibrated layout.**
   Prefer `device.pre_calibrated_layouts`; a brand-new layout needs fresh calibration, which takes time.
3. **Stay inside the limits:**
   durations in multiples of the clock period, total under 6 µs, amplitude and detuning within the channel maxima.
4. **Emulate with noise**
   (`prefer_device_noise_model=True`) so you know what "good" looks like on noisy hardware.
5. **Dry-run with `mimic_qpu=True`**
   on an emulator backend.
6. **Choose shots deliberately.**
   At most 500 per job on Orion devices; more shots shrink statistical error, but cost more.
7. **Record the batch ID**
   and compare QPU counts with your emulated baseline.

CHECK YOURSELF · PART 6

tap a question to reveal the answer

Q You need to emulate 40 atoms in a 2D array. Which emulator?

EMU-MPS (or emu-mps locally). Exact state-vector methods top out around 25 atoms. Check that the bond dimension doesn't hit its cap.

Q What is the difference between a job and a run?

A run (shot) is one execution that yields one bitstring. A job is a set of runs with the same variable values.

Q Your open batch stopped with TIMED_OUT. Why?

No new job arrived for a few minutes, so the reserved backend was released. Make your classical processing step faster, or submit the next job sooner.

Q Can you run on FRESNEL with the free Explorer offer?

No. Explorer gives EMU-FREE only. QPU access needs a contract and credits, or pay-as-you-go through Google Cloud Marketplace or another partner cloud.

Q Why can't today's cloud QPUs run per-qubit logic gates?

They expose a single global Rydberg channel. Every atom receives the same pulse, so these devices operate in analog mode.

PART 7 · HIGHER-LEVEL TOOLS

Solvers, QoolQit and an AI co-pilot

Pulser gives you complete control, but you don't always want it. Pasqal's higher-level libraries handle the physics for you: you describe a problem, and they choose atom positions, design pulses, run them and interpret the results. All of them produce Pulser sequences underneath, so what you learned in Part 5 still applies.

YOU WILL LEARN

- QoolQit and "unit-free" analog programming
- The MIS, QUBO and QEK libraries
- The Neutral Atom Toolkit for AI coding agents
- What happened to Qadence

TIME

- About 40 minutes to read
- An afternoon to try the quickstarts

7.1 QoolQit: analog programs without units

QoolQit is a Python library for designing analog programs in what Pasqal calls the **Rydberg Analog Model**. Its central idea is **adimensionalisation**: you write programs without hardware units (no µm, ns or rad/µs), which makes them independent of any particular device. A separate compilation step then converts your unit-free program into physical numbers that fit a chosen device.

That separation is useful. The problem you are solving (a graph, a pattern of weights) stays clean, and the details of one machine's spacing or laser limits are handled by the compiler. QoolQit is actively maintained and is the recommended starting point for new analog algorithm work (current docs version: 1.4.0).

python · from the QoolQit docs

```python
# pip install qoolqit
from qoolqit import ConstantWaveform, Drive, InterpolatedWaveform, RampWaveform

# Waveforms take a duration first (unit-free), and chain with >>
amplitude = ConstantWaveform(5.0, value=1.0) >> InterpolatedWaveform(10.0, [0.0, 0.8, 0.8, 0.0])
detuning  = RampWaveform(8.0, -1.0, 1.0) >> ConstantWaveform(4.0, 1.0)

drive = Drive(amplitude=amplitude, detuning=detuning)   # omitted detuning/phase default to zero
drive = drive >> drive                                  # drives compose too
drive.draw()
```

Key objects

`Register` and `DataGraph` (where atoms go, and the graph they represent), waveforms, `Drive` (amplitude + detuning + phase over time), `QuantumProgram(register, drive)`, devices, and **embedders** that turn data into registers.

Workflow

Build a register and a drive, combine them into a `QuantumProgram`, compile it for a target device (which produces a Pulser sequence), then execute it on an emulator or QPU.

Its docs are organised as Get started, Fundamentals (graphs, embedding, writing, compiling and executing programs), Extended usage (adimensionalisation, custom embedders) and Applications (solving a QUBO, solving a MWIS problem, preparing the critical state of the transverse-field Ising model). At the time of writing, Pasqal is also running a QoolQit Contest on its community site, with QPU credits as prizes.

> WATCH OUT
>
> QoolQit is moving fast and its API pages sometimes differ between versions (for example, a `dmm` argument in older reference pages versus `weighted_detunings` in newer ones). Match the docs version to the version you installed.

7.2 The MIS solver

The `maximum-independent-set` library compiles MIS problems into sequences for analog hardware. You don't need any quantum knowledge to try it: hand it a graph and get back optimal or near-optimal independent sets. Advanced users can tune the strategy, run on Pasqal Cloud, or build custom pipelines.

python · quickstart from the docs

```python
# pip install maximum-independent-set
from mis import MISSolver, MISInstance, BackendConfig, SolverConfig
import networkx as nx

graph = nx.Graph()
graph.add_edges_from([(0, 1), (0, 2)])     # a path of three nodes: 1 - 0 - 2
instance = MISInstance(graph)

config = SolverConfig(backend=BackendConfig())   # default: a local quantum emulator
solver = MISSolver(instance, config)
results = solver.solve()

print("MIS solutions:", results)              # the best answer is nodes 1 and 2 (the two ends)
results[0].draw()
```

The tutorials cover weighted MIS, programming the quantum part yourself, working around device constraints, and solving MIS through a QUBO formulation. Graph colouring lives in the same package (`mis.coloring`).

7.3 The QUBO solver

The `qubo-solver` library takes a symmetric matrix Q and searches for the binary vector that minimises the QUBO cost from Part 3. It offers **quantum**, **classical** and **hybrid** solvers, so you can compare them on the same instance.

python · quickstart from the docs

```python
# pip install qubo-solver
from qubosolver import Instance, Solver, matrix, analysis

Q = matrix.tensor([
    [-0.2, 0.0, 1.0],
    [ 0.0, -1.0, 1.5],
    [ 1.0, 1.5, -0.1],
])
instance = Instance(Q)

solver = Solver(instance)        # the quantum solver is the default
solution = solver.solve()
print(analysis.to_dataframe([solution]))
```

Embedding

How the matrix becomes atom positions: **Greedy Layout** and **BLaDE**.

Drive shaping

How the pulse is designed: **Proportional-Diagonal**, **Local-Energy-Scale**, **Bayesian Search**, or your own.

7.4 QEK: the quantum evolution kernel library

`quantum-evolution-kernel` implements the graph-similarity method from Part 3 for the machine-learning community. A typical workflow: load a molecule dataset (the tutorials use PTC-FM, a toxicity benchmark, via PyTorch Geometric), compile each graph into a register with a compiler class, run the extractor on an emulator or QPU to get bitstring distributions, then build a `QuantumEvolutionKernel(mu=...)` and train a scikit-learn `SVC` on it.

The method comes from the paper *Quantum feature maps for graph machine learning on a neutral atom quantum processor* (Physical Review A 107, 042615; arXiv 2107.03247). Install with `pip install quantum-evolution-kernel`; the tutorials walk from feature extraction to a trained classifier.

7.5 The Neutral Atom Toolkit: an AI co-pilot for experiments

Launched on 8 October 2026, the **Neutral Atom Toolkit** (also called the Neutral-Atom Agentic Toolkit) is a set of **skills** for AI coding agents such as Claude Code, Codex, Gemini CLI, Cursor and GitHub Copilot. Each skill is a set of instructions plus tested Python scripts. Together they automate the machine-facing steps of an experiment, while you keep the scientific decisions and approve anything that spends money.

Claude Code

```
/plugin marketplace add pasqal-io/agent-marketplace
/plugin install neutral-atom-toolkit@pasqal
```

SKILL	WHAT IT DOES
`application-to-idea` (optional)	Turns an application or problem into a documented experimental method

SKILL	WHAT IT DOES
`idea-to-spec`	Turns a description, paper or patent into an experiment specification
`spec-to-sequence`	Writes a Pulser sequence file that tests itself
`validate-emu`	Runs noiseless and noisy emulation, then gives a go/no-go for the QPU
`noise-emulate` (optional)	Noise emulation locally, on a GPU cluster, or on Pasqal Cloud
`qpu-submit` / `submit-to-cea`	Submits to a cloud QPU, or to the Ruby QPU at CEA, after you approve the shot count
`harvest-and-analyze`	Collects results and compares them with the emulated baseline

Every step writes plain files (a JSON spec, a Python sequence, raw bitstrings), so you can inspect and rerun anything. Local steps need only Python and Pulser, with no account. Cloud steps read `PASQAL_USERNAME`, `PASQAL_PASSWORD` and `PASQAL_PROJECT_ID` from environment variables, and nothing is submitted without your explicit approval of the shot count. The approach is described in the paper *Lowering the implementation barrier of neutral-atom quantum computing with agentic workflows* (arXiv 2607.25834, July 2026).

> KEY IDEA · USE IT AFTER YOU UNDERSTAND THE BASICS
>
> An agent that writes Pulser code is most useful when you can read and judge that code. The paper behind the toolkit notes that human review caught real mistakes, including an inadequate observable. Work through Parts 5 and 6 first, then let the toolkit speed you up.

7.6 Qadence and other libraries you may meet

Qadence was Pasqal's library for **differentiable digital-analog** programs: you composed circuits from "blocks" (`chain` for sequence, `kron` for parallel, `add` for sums), mixed digital gates with analog blocks such as `AnalogRX` and `AnalogInteraction`, and trained them like PyTorch neural networks. It appears in many 2023–24 tutorials and papers. Its GitHub page now states that it is **not actively maintained**, and it has left the main docs hub; its documentation survives at pasqal-io.github.io/qadence. Read it to understand digital-analog ideas, but start new projects with Pulser or QoolQit.

LIBRARY	WHAT IT IS	STATUS (OCT 2026)
`emu-sv`, `emu-mps`	Local state-vector and tensor-network emulators	Very active
`qoolqit`	Unit-free analog programming	Very active
`pasqal-cloud`	Cloud SDK	Very active
`maximum-independent-set`, `qubo-solver`, `quantum-evolution-kernel`	Application libraries	Active
`pulser-azure`, `Pulser-myQLM`	Integrations with Azure Quantum and the myQLM framework	Active

LIBRARY	WHAT IT IS	STATUS (OCT 2026)
`qadence`	Digital-analog, differentiable programming	Not actively maintained
`pyqtorch`, `horqrux`	PyTorch and JAX state-vector simulators (Qadence backends)	Low activity

PART 8 · PASQAL, THE COMPANY

Where Pasqal came from and where it is going

Knowing the company's story helps you read its documentation, papers and announcements in context: why analog mode came first, why the machines sit in supercomputing centres, and which roadmap targets are achievements versus plans.

8.1 From a physics lab to Nasdaq

Pasqal grew out of decades of research at the **Institut d'Optique** (Laboratoire Charles Fabry) in Palaiseau, south of Paris. Its official founders are **Georges-Olivier Reymond**, **Christophe Jurczak**, **Antoine Browaeys** and **Thierry Lahaye**. **Alain Aspect**, whose 1982 experiments on entanglement earned him a share of the 2022 Nobel Prize in Physics, is widely named as a co-founder, and his work opens the company's own history page.

1982	Alain Aspect's experiment demonstrates the violation of Bell's inequalities, a landmark test of entanglement.
2001	Georges-Olivier Reymond traps a single atom in an optical tweezer (*Nature*).
2009	Antoine Browaeys's group demonstrates the first Rydberg blockade between two individual atoms (*Nature Physics*).
2016	Lahaye and Browaeys run a quantum simulation with 30 atoms (*Nature*).
2018	Simulations with 49 qubits, and 72 atoms arranged in 3D.
2019	Pasqal is founded as a spin-off of the Institut d'Optique.
2021	A *Nature* paper on scalable quantum simulation lays the basis for the commercial 100+ qubit systems. The EuroHPC-funded HPCQS project starts (December).
Jan 2022	Merger with **Qu&Co**, an Amsterdam quantum-algorithms company, bringing software and industrial customers.
Jan 2023	€100 million Series B led by Temasek. Pulser Studio launches as an open beta.
2024	Satellite planning with Thales (May), an agreement with Aramco (May), more than 1,000 atoms trapped (June), EV smart charging with EDF (September), first QPU delivered to GENCI/CEA.
Jun 2025	New technology roadmap, with a path to fault-tolerant computing.
Oct 2025	The Canadian QPU (FRESNEL_CAN1) goes live on Pasqal Cloud.
Nov 2025	The **Jade** (Jülich) and **Ruby** (CEA/TGCC) QPUs are inaugurated as Europe's first hybrid HPC-quantum infrastructure. Aramco and Pasqal announce Saudi Arabia's first quantum computer, a 200-qubit system.

Apr 2026	Defect-free registers of **1,024 atoms**, doubling the previous record of 506.
Aug 2026	Pasqal lists on **Nasdaq** (ticker **PSQL**) through a merger with the SPAC Bleichroeder Acquisition Corp. II, with about $360 million in cash at closing.
Oct 2026	Pasqal leads **Q-PLANET**, a €50 million EU pilot line for neutral-atom chip production (28 partners), and launches the Neutral Atom Toolkit.

Leadership (October 2026)

Wasiq Bokhari is CEO (since March 2026, after serving as executive chairman). Loïc Henriet, previously CEO and lead author of the field's standard review paper, is CTO.

Size and offices

Headquarters in Palaiseau, France, with regional hubs in Riyadh, Boston, Sherbrooke (Canada) and Seoul. Around 275–300 employees; the website cites 85 patents and 70 PhDs.

8.2 Where the machines are

Pasqal's strategy puts physical machines inside supercomputing centres and at industrial sites, not only in its own lab. When it listed on Nasdaq, the company reported **seven QPUs deployed, three of them in production**.

France · Pasqal and CEA/TGCC

FRESNEL (cloud) and Ruby at the TGCC computing centre, funded through HPCQS, a €12 million EuroHPC project coordinated by Forschungszentrum Jülich.

Germany · Jülich

Jade, integrated with the JSC supercomputers. Users run hybrid jobs with ordinary HPC tools such as Slurm.

Italy · CINECA

Sol, an Orion Gamma machine (140 qubits) designed to work with the Leonardo supercomputer.

Canada and Saudi Arabia

FRESNEL_CAN1 on Pasqal Cloud; FRESNEL_SA1, a 200-qubit-class system in Aramco's Dhahran data centre, available through the Saudi cloud region.

8.3 The roadmap

Pasqal's June 2025 roadmap names a hardware generation for each year:

GENERATION	TARGET YEAR	PHYSICAL QUBITS	FOCUS
Orion Gamma	end of 2025	140+	Analog; in production today
Vela	2027	200+	Higher repetition rate and fidelity, parallel gates
Centaurus	2028	not stated	Early fault-tolerant computing
Lyra	2029	10,000–50,000 (2026 filings)	"Impactful" fault-tolerant computing

Pasqal's stated logical-qubit targets rise from 2 (2025) to 20 (2027), 100 high-fidelity logical qubits (2029) and 200 (2030). Its 2026 filings describe the aim as 10,000+ physical and 200+ logical qubits at 99.9999% fidelity in 2029.

WATCH OUT · TARGETS VS RESULTS

Roadmaps move. A 2024 version promised 10,000 physical qubits in 2026; that has been replaced by the plan above. The 1,024-atom result of April 2026 means 1,024 trapped atoms, not 1,024 working computational qubits. The 2025 roadmap also targeted a quantum-advantage demonstration in the first half of 2026; when this guide was written, no primary Pasqal announcement or paper confirming one had been found. Always check the dates on claims, and read Pasqal's newsroom for the latest.

Physical vs logical qubits

A physical qubit is one atom. A logical qubit is one reliable qubit made from many physical ones using quantum error correction. Error-correction codes detect and fix errors without reading the stored information directly.

Fault tolerance

If physical error rates fall below a threshold, adding more physical qubits per logical qubit makes logical errors arbitrarily small. "Fault-tolerant" means the whole machine, correction included, works below that threshold.

8.4 Who uses it, and for what

Pasqal reports 25+ commercial use cases across 35+ customer engagements. Some public examples:

PARTNER	SECTOR	PROBLEM	METHOD FAMILY
EDF	Energy	Smart charging for large electric-vehicle fleets (announced Sept 2024)	Optimisation
Thales	Aerospace	Satellite mission planning (AQUAPS project, May 2024)	Optimisation / MIS
CMA CGM	Shipping	Container management and ship loading; a quantum centre of excellence at its TANGRAM training centre	Optimisation
Aramco	Energy / industry	Port logistics, CO_2 storage, well placement, rig scheduling	Optimisation
Crédit Agricole CIB	Finance	Portfolio optimisation, derivatives pricing, fraud detection	Optimisation, ML
Qubit Pharmaceuticals	Pharma	Predicting where water molecules sit in proteins; winners of Wellcome Leap's Quantum for Bio programme	Simulation / algorithms
Pasqal research	Chemistry	Predicting molecular toxicity with QEK on a real QPU	Graph ML

Notice the pattern: most industrial work maps onto **MIS or QUBO** optimisation, or onto **graph learning**. These are the problems where the blockade and the freedom to place atoms anywhere are most natural.

8.5 Neutral atoms compared with other qubits

Each hardware approach makes different trade-offs. Cross-technology comparisons are always simplified, and the non-neutral-atom columns below show typical ranges from the general literature, not any vendor's specification.

	NEUTRAL ATOMS (PASQAL)	SUPERCONDUCTING	TRAPPED IONS	PHOTONICS
Qubit	One rubidium atom in a light trap	A tiny circuit on a chip	One charged atom in an electric trap	A particle of light
Cooling	Atoms at μK; machine at room temperature	Whole chip at ~10–15 mK in a dilution fridge	Ions laser-cooled; room-temperature chamber	Mostly room temperature; detectors often cryogenic
Connectivity	Set by geometry; reconfigurable; atoms can be moved	Fixed nearest neighbours on the chip	All-to-all within a chain	Flexible, but entangling is probabilistic
Speed	Fast pulses, slow shots (~0.25–5 per second)	Fastest gates, thousands of shots per second	Slower gates	Limited by photon loss and success rates
Scale today	100–200 qubits per QPU; 1,000+ atoms in research	Hundreds to ~1,000 qubits	Typically tens to ~100	Large in special-purpose sampling
Strength	Identical qubits, flexible layouts, native graph problems	Mature fabrication, speed	Best fidelity and coherence	Networking, room temperature
Weakness	Slow repetition, atom loss, global-only control today	Cryogenics, wiring, short coherence	Hard to scale one trap	Loss, probabilistic gates

Other neutral-atom companies include **QuEra** (USA, analog machine available on AWS Braket), **Atom Computing** (USA, strontium atoms, partnered with Microsoft), **Infleqtion** (USA, caesium, also builds clocks and sensors) and **planqc** (Germany, strontium, HPC-oriented). Pasqal stands out for the number of machines physically installed in HPC centres and industrial sites, and for its European position.

PART 9 · KEEP GOING

A study plan, a cheat sheet and where to go next

You have seen the whole platform once. This part turns it into a routine: a six-week plan that pairs this guide with Pasqal's own course and tutorials, a one-page cheat sheet, a searchable glossary, and the links you will keep coming back to.

9.1 A six-week study plan

Plan on four to six hours a week. Each week ends with a concrete "done when" check, so you know when to move on.

WEEK 1

Quantum foundations

- This guide, Part 1
- Quantum Quest modules 1–2 (Introduction, Mathematics)
- Practise: compute probabilities from amplitudes by hand

Done when you can explain superposition, shots and entanglement to a friend without notes.

WEEK 2

The atom machine

- This guide, Part 2, plus Labs 1 and 2
- Quantum Quest module 3 (Physics)
- Skim the Henriet et al. review, sections 1–3

Done when you can work out the blockade radius for a given Ω and say whether two atoms block each other.

WEEK 3

First Pulser programs

- Install Pulser; this guide, 5.1–5.9
- Pulser tutorial "Programming with Pulser"
- Pulser Studio tutorial 1 (Rabi oscillations)

Done when you have plotted a Rabi oscillation by sweeping a pulse's duration in a loop.

WEEK 4

Analog algorithms

- This guide, Part 3 and 5.10–5.13, plus Lab 3
- Tutorials: AFM state, QAA for QUBO, QAA for MWIS
- Add a NoiseModel and compare results

Done when your AFM sweep makes the checkerboard the top bitstring, and you can explain what noise does to it.

WEEK 5

Emulators and the cloud

- This guide, Part 6
- Create a Pasqal Cloud account; run a job on EMU-FREE
- Install emu-mps; run a 20+ atom register locally

Done when you have a batch ID, downloaded results, and a passing `mimic_qpu=True` dry run.

WEEK 6

Applications and a project

- This guide, Part 7; Quantum Quest modules 4–5
- Run the MIS solver on a real graph (a small road map, a schedule)
- Try the QoolQit tutorial

Done when you have written up one mini-project: problem, encoding, pulse, emulated results, and what you'd run on a QPU.

KEY IDEA · LEARN IN PUBLIC

Pasqal's community Slack has over 700 members, and Pasqal regularly runs hackathons, coding challenges and beta-testing programmes. Asking a question there, or entering a challenge, is often the fastest way to get unstuck and to get real QPU credits.

9.2 Cheat sheet

Units in Pulser

Time in **ns** (integers). Ω and δ in **rad/µs**. Distances in **µm**. Phase in rad. MHz × 2π = rad/µs. 1000 ns at Ω = π rad/µs is a π-pulse.

Bits

Atoms start in |g⟩. ground-rydberg basis: |r⟩ reads **1**, |g⟩ reads **0**. Bit order = register order.

Core formulas

Rabi: $P_r = \Omega^2/(\Omega^2+\delta^2) \cdot \sin^2(\sqrt{(\Omega^2+\delta^2)}\cdot t/2)$. Pulse area $\theta = \int\Omega\, dt$. Interaction $U = C_6/R^6$. Blockade radius $R_b = (C_6/\Omega)^{1/6}$. N blockaded atoms oscillate at $\sqrt{N}\cdot\Omega$.

Hamiltonian

$H = \Sigma\, (\Omega/2)(\cos\varphi\, \sigma^x - \sin\varphi\, \sigma^y) - \Sigma\, \delta\, n_i + \Sigma\, C_6/R_{ij}^6\, n_i n_j$. Drive = mixer, detuning = reward, interaction = penalty.

Orion QPU limits (docs)

100 (Beta) / 200 (Gamma) atoms · min spacing 5 µm · max radius 35 / 70 µm · Rydberg n = 60 · C_6 = 865,723 rad·µm⁶/µs · sequence ≤ 6,000 ns · ≤ 500 shots/job · layout ≤ 200 traps, ≤ 45% filled · global Rydberg channel only.

Channel rules (AnalogDevice)

$\Omega \le 12.57$ rad/µs · $|\delta| \le 125.66$ rad/µs · durations multiple of 4 ns, ≥ 16 ns · R_b at max $\Omega \approx 6.4$ µm.

Emulator sizes

EMU-FREE ~10 atoms (free) · EMU-SV ~25 (exact) · EMU-MPS up to 80 (approximate) · QPU beyond that.

Cloud words

Run = shot = one bitstring. Job = runs with fixed variables. Batch = ≤1,000 jobs, one backend. Project = team space with an ID that pays.

python · the skeleton of every program

```python
import numpy as np, pulser

reg = pulser.Register.square(3, spacing=7.0, prefix="q")          # 1. where the atoms sit
seq = pulser.Sequence(reg, pulser.AnalogDevice)                    # 2. the rules
seq.declare_channel("ryd", "rydberg_global")                       # 3. the laser
seq.add(pulser.Pulse.ConstantDetuning(                             # 4. the pulses
    pulser.BlackmanWaveform(1000, np.pi), 0.0, 0.0), "ryd")
results = pulser.backends.QutipBackendV2(seq).run()                # 5. run (emulator)
print(results.final_bitstrings.most_common(5))                     # 6. read the counts

# On the cloud: from pasqal_cloud import PasqalCloudConnection, RemoteEmuFreeBackend
#               from pulser.backends import QPUBackend
```

9.3 Glossary

Pasqal's documentation has no single glossary page, so here is one. Search, or filter by topic.

Qubit — QUANTUM

A two-level quantum system; on Pasqal machines, one atom using two of its energy levels.

Superposition — QUANTUM

A qubit state that combines both levels at once, described by two amplitudes.

Amplitude — QUANTUM

A complex number (size and phase) for each possibility; its squared size is the probability.

Phase — QUANTUM

The "direction" part of an amplitude, invisible in one measurement but essential for interference. Also the laser's φ.

Measurement
QUANTUM

Reading qubits; it returns one outcome and destroys the superposition. Destructive on neutral-atom QPUs.

Bitstring
QUANTUM

One measured 0/1 per atom, in register order; the output of a single shot.

Bloch sphere
QUANTUM

A sphere on which every single-qubit state is a point; pulses rotate the state around it.

Entanglement
QUANTUM

A shared state of several qubits that can't be split into separate states for each.

Interference
QUANTUM

Amplitudes adding or cancelling like waves; the source of quantum speed-ups.

Hamiltonian
QUANTUM

The energy function of a system, which also dictates how its state evolves in time.

Ground state
QUANTUM

The lowest-energy state of a Hamiltonian; in optimisation, the encoded answer. Also the atom's lowest level $|g\rangle$.

Schrödinger equation
QUANTUM

The equation that tells you how a quantum state changes under a Hamiltonian.

Quantum phase transition
QUANTUM

A sudden change in a system's ground-state pattern as a parameter is tuned, driven by quantum effects.

Logical qubit
QUANTUM

One error-protected qubit built from many physical qubits using error correction.

Fault tolerance (FTQC)
QUANTUM

Computing reliably despite errors, by keeping physical error rates below an error-correction threshold.

NISQ
QUANTUM

Noisy Intermediate-Scale Quantum: today's era of useful but error-prone machines with tens to hundreds of qubits.

Quantum advantage
QUANTUM

Beating the best classical method on a specific, well-defined problem. Always problem-specific.

Neutral atom
HARDWARE

An uncharged atom (rubidium-87 at Pasqal) used as a qubit.

Optical tweezer
HARDWARE

A tightly focused laser spot (~1 μm) that traps a single atom.

SLM
HARDWARE

Spatial light modulator: a programmable hologram that creates a pattern of fixed tweezers.

AOD
HARDWARE

Acousto-optic deflector: steers a moving tweezer used to rearrange atoms.

MOT
HARDWARE

Magneto-optical trap: lasers plus a magnetic field that cool a cloud of atoms to μK.

Rearrangement
HARDWARE

Moving atoms from randomly loaded traps into the requested register; succeeds >99%.

Rydberg state
HARDWARE

A highly excited atomic state (n = 60 on Pasqal QPUs) with very strong interactions.

Hyperfine qubit
HARDWARE

Two closely spaced ground levels ($|g\rangle$, $|h\rangle$): long-lived, used for digital gates via Raman lasers.

Rabi frequency (Ω)
HARDWARE

The laser drive strength, setting how fast an atom oscillates between levels. In rad/μs.

Detuning (δ)
HARDWARE

The laser's frequency offset from the atomic transition; acts as a reward or cost for excitation.

Rabi oscillation
HARDWARE

The back-and-forth swing of an atom between $|g\rangle$ and $|r\rangle$ under a constant drive.

π-pulse
HARDWARE

A pulse of area π that fully flips $|g\rangle$ to $|r\rangle$ (on resonance). A π/2-pulse makes a 50/50 state.

Van der Waals interaction
HARDWARE

The repulsion C_6/R^6 between two Rydberg atoms.

C_6 coefficient

HARDWARE

The strength constant of the interaction; 865,723 rad·μm⁶/μs for n = 60.

Rydberg blockade

HARDWARE

Close atoms can't both be excited, because the interaction pushes |rr⟩ out of resonance.

Blockade radius

HARDWARE

$R_b = (C_6/\Omega)^{\wedge}(1/6)$: atoms closer than this block each other. About 6.4 μm at full power.

Global addressing

HARDWARE

One laser beam drives every atom identically; the only mode on today's cloud QPUs.

Local addressing

HARDWARE

Driving only selected atoms with a steered beam.

Fluorescence imaging

HARDWARE

Photographing glowing atoms to see which traps are full; dark sites after a sequence read as 1.

SPAM errors

HARDWARE

State preparation and measurement errors: missing atoms, false positives, false negatives.

Dephasing

HARDWARE

Random drift of the phase between qubit levels, which washes out quantum effects.

Orion

HARDWARE

Pasqal's QPU family: Alpha (Fresnel), Beta (100 qubits), Gamma (140–200 qubits).

Repetition rate

HARDWARE

Shots per second; low on neutral atoms (Pasqal Cloud estimates 4 s per shot) because atoms are reloaded every time.

Analog computing

ALGORITHMS

Programming by shaping a Hamiltonian's evolution over time instead of applying gates.

Digital computing

ALGORITHMS

Gate-based computing: discrete operations on chosen qubits.

Digital-analog (DAQC)

ALGORITHMS

Mixing digital gates with analog evolution blocks.

CZ gate

ALGORITHMS

Controlled-Z, a two-qubit gate built from the blockade with a π, 2π, π pulse sequence.

Adiabatic theorem

ALGORITHMS

A slowly changed system stays in its instantaneous ground state.

QAA

ALGORITHMS

Quantum Adiabatic Algorithm: sweep δ slowly from negative to positive so atoms end in the problem's ground state.

Minimum gap

ALGORITHMS

The smallest energy difference between ground and first excited state during a sweep; sets how slow the sweep must be.

QAOA

ALGORITHMS

Quantum Approximate Optimisation Algorithm: alternating evolutions whose durations are tuned by a classical optimiser.

Variational algorithm

ALGORITHMS

A loop in which a classical optimiser tunes quantum parameters using measured results.

Independent set

ALGORITHMS

A set of graph nodes with no edge between any two of them.

MIS / MWIS

ALGORITHMS

Maximum (weighted) independent set: the largest (or heaviest) independent set. Native to the blockade.

Unit-disk graph

ALGORITHMS

Nodes are points; edges join points closer than a fixed distance. Exactly what a register with a blockade radius is.

QUBO

ALGORITHMS

Quadratic Unconstrained Binary Optimisation: minimise $\Sigma\, Q_{ij}\, z_i\, z_j$ over binary z_i.

Embedding

ALGORITHMS

Choosing atom positions (and detunings) so the hardware's energy reproduces a problem.

Antiferromagnetic (AFM) state

ALGORITHMS

A checkerboard pattern of excited and ground atoms; the classic analog demo.

Ising model

ALGORITHMS

The textbook model of interacting magnetic spins; the Rydberg Hamiltonian is a version of it.

QEK
ALGORITHMS

Quantum Evolution Kernel: a graph-similarity measure from QPU fingerprints, used in classical ML.

State-vector emulation
ALGORITHMS

Exact simulation storing all 2^N amplitudes; practical to ~25 atoms.

Matrix product state (MPS)
ALGORITHMS

An approximate chain-of-tensors representation of a quantum state; cheap when entanglement is low.

Bond dimension
ALGORITHMS

The size of the links in an MPS; how much entanglement it can capture.

Pulser
SOFTWARE

Pasqal's open-source Python library for pulse-level programming of neutral-atom devices.

Register
SOFTWARE

A named set of atom positions (µm). Its order sets the bit order.

Register layout
SOFTWARE

All trap positions a machine provides; registers choose traps from it. Required on QPUs.

Mappable register
SOFTWARE

A register whose trap assignment is chosen at build time; lets one batch use different registers.

Device
SOFTWARE

The machine's rules: atom count, spacing, channels and their limits.

VirtualDevice / MockDevice
SOFTWARE

Configurable or unconstrained devices for learning and prototyping.

Channel
SOFTWARE

One controllable laser or microwave source, such as `rydberg_global`.

DMM
SOFTWARE

Detuning map modulator: adds weighted extra detuning to chosen atoms.

SLM mask
SOFTWARE

Detunes chosen atoms far away so a global pulse skips them.

EOM mode
SOFTWARE

A channel mode giving sharper square pulses.

Pulse
SOFTWARE

Amplitude waveform + detuning waveform + phase over one duration.

Waveform
SOFTWARE

A shape over time: Constant, Ramp, Blackman, Kaiser, Interpolated, Composite, Custom.

Sequence
SOFTWARE

A full Pulser program: register + device + channels + pulses, validated as you build it.

Parametrized sequence
SOFTWARE

A sequence with variables, built later with concrete values via `build()`.

Modulation bandwidth
SOFTWARE

How quickly a channel can change; it smooths programmed pulses into real ones.

Backend
SOFTWARE

What runs a sequence: `QutipBackendV2`, `SVBackend`, `MPSBackend`, remote emulators or `QPUBackend`.

Observable
SOFTWARE

A quantity an emulator computes: BitStrings, Occupation, Energy, Fidelity, CorrelationMatrix…

NoiseModel
SOFTWARE

Pulser's object for switching on realistic noise in emulation.

emu-sv / emu-mps
SOFTWARE

Pasqal's local state-vector and matrix-product-state emulators.

QoolQit
SOFTWARE

Library for unit-free ("adimensionalised") analog programs, compiled to devices.

Pulser Studio
SOFTWARE

No-code, in-browser editor for registers and pulses that also generates Pulser code.

Qadence
SOFTWARE

Former digital-analog, differentiable programming library; no longer actively maintained.

Neutral Atom Toolkit SOFTWARE
AI-agent skills that draft, emulate and submit experiments, with human approval.

QPU CLOUD
Quantum Processing Unit: the physical machine that runs sequences.

Pasqal Cloud CLOUD
Pasqal's hosted service for QPUs and remote emulators, with a web portal and Python SDK.

pasqal-cloud CLOUD
The Python package; `PasqalCloudConnection` is its entry point.

Project CLOUD
A collaboration space with an ID; required to submit jobs, and it determines billing.

Batch CLOUD
Up to 1,000 jobs sharing one backend and (usually) one sequence.

Open batch CLOUD
A batch that accepts new jobs while running, for variational loops; times out if idle.

Job CLOUD
A number of runs of one sequence with fixed variable values.

Run / shot CLOUD
One execution returning one bitstring. Also called a repetition.

EMU-FREE / EMU-SV / EMU-MPS CLOUD
Pasqal Cloud emulators: free QuTiP (~10 atoms), exact state vector (~25), tensor network (up to 80).

mimic_qpu CLOUD
An emulator option that applies a real QPU's validation checks.

Credits CLOUD
Prepaid units in pools (valid one year by default) that pay for emulator hours and QPU shots.

Explorer offer CLOUD
Pasqal Cloud's free tier: EMU-FREE only, up to 10 concurrent jobs, low priority.

Third-party clouds CLOUD
Google Cloud, Azure Quantum, OVHcloud and Scaleway, which bill pay-as-you-go.

9.4 Resources

Official documentation and tools

Pasqal documentation hub
Every product, plus onboarding and product news.
`docs.pasqal.com`

Pulser documentation
Concepts, conventions, tutorials and API reference.
`pulser.readthedocs.io`

Pulser on GitHub
Source code and every tutorial notebook, including retired ones.
`github.com/pasqal-io/Pulser`

QoolQit
Unit-free analog programming.
`docs.pasqal.com/qoolqit`

QPUs and emulators
Device specs and the local emulators.
`docs.pasqal.com/qpu-emulators`

Pasqal Cloud docs
Accounts, first job, batches, billing.
`docs.pasqal.com/cloud`

Pasqal Cloud portal
Sign up, join projects, track jobs.
`portal.pasqal.cloud`

Pulser Studio
The no-code editor.
`pulserstudio.pasqal.cloud`

Quantum applications

MIS, QUBO and QEK libraries.

`docs.pasqal.com/applicationsolvingtools`

Neutral Atom Toolkit

Agent skills for running experiments.

`docs.pasqal.com/agentic-toolkit`

Courses and community

Quantum Quest

Pasqal's interactive course: Introduction, Mathematics, Physics, Optimization, Machine Learning.

`community.pasqal.com/learning`

Pasqal Community

Learning, events, challenges and tools in one place.

`community.pasqal.com`

Community Slack

Questions, peer help and announcements.

`insights.pasqal.com/community_joinslack`

Pasqal on YouTube

Webinar recordings and talks.

`youtube.com/@pasqal.quantum`

Pasqal blog

Explainers and research highlights.

`pasqal.com/blog`

Product news

What changed in the software and devices.

`docs.pasqal.com/productnews`

Papers worth reading

Quantum computing with neutral atoms

Henriet et al., *Quantum* 4, 327 (2020). The standard review; start here.

`arXiv 2006.12326`

Pulser: an open-source package...

Silvério et al., *Quantum* 6, 629 (2022). The design of Pulser.

`arXiv 2104.15044`

Quantum feature maps for graph ML

Phys. Rev. A 107, 042615. The QEK method.

`arXiv 2107.03247`

Agentic workflows for neutral atoms

Dalyac, Dauphin, Henriet, Jurczak (2026). The paper behind the Neutral Atom Toolkit.

`arXiv 2607.25834`

Company

pasqal.com

Technology, solutions and use cases.

`pasqal.com`

Newsroom

Announcements, partnerships and results.

`pasqal.com/newsroom`

Neutral-Atom Field Guide · an independent learning guide to the Pasqal platform.

Not an official Pasqal publication, and not affiliated with or endorsed by Pasqal. "Pasqal", "Pulser", "Orion" and other product names belong to their owners. The colour palette is inspired by Pasqal's public website to make the guide feel familiar to its users.

Fact-checked October 2026 against docs.pasqal.com, pulser.readthedocs.io, pasqal.com, Pasqal's Orion Beta technical overview, the Henriet et al. review, and Pulser 1.9.1 / pasqal-cloud 0.24.0 installed locally. Code marked "tested" was run on that install.

Spotted something out of date? Check the official docs first; they take precedence.